



《计算机组成与设计》课程设计报告

目录

一、简单的模型机设计概述.....	4
1.1 设计目的.....	4
1.2 设计步骤.....	4
1.3 拟定指令系统.....	4
1.4 确定总体结构.....	5
1.5 逻辑设计.....	5
1.6 确定控制方式.....	5
1.6.1 微程序实现控制器.....	5
1.6.2 硬布线实现控制器.....	6
二、课程设计环境.....	6
2.1 硬件环境.....	6
2.2 软件环境.....	7
2.2.1 QuartusII.....	7
2.2.2 计算机组成原理与系统结构.....	7
三、指令组.....	7
3.1 微程序实验指令组.....	7
3.1.1 基本字长.....	7
3.1.2 指令格式.....	7
3.1.3 指令类型.....	8
3.1.4 寻址方式.....	10
3.2 硬布线实验指令组.....	10
四、总体结构与分部件结构.....	11
4.1 设计概述.....	11
4.1.1 寻址方式.....	11
4.1.2 设计简述.....	11
4.2 设计概述.....	11
4.2.1 微程序设计总体框图.....	12
4.2.2 硬布线设计总体框图.....	12
4.3 寄存器.....	12
4.4 运算器.....	15
4.5 选择器.....	18
4.6 UPC 设计.....	20
4.7 译码器.....	21
4.8 D 触发器节拍发生器.....	22

4.9 开关装置（即启动电路）	23
五、微程序实现的模型机内核.....	23
5.1 确定模型机的功能.....	23
5.2 微程序实现的 CU 框图及详解.....	24
5.2.1 下地址形成电路.....	24
5.2.2 控制存储器 ROM.....	26
5.2.3 译码器.....	27
5.2.4 RAM.....	27
5.3 设计总体结构与数据通路.....	28
5.4 微程序控制时序.....	30
5.5 微指令格式.....	31
5.6 指令执行流程.....	32
5.7 编制微程序.....	43
5.8 完整电路结构.....	46
六、硬布线实现的模型机内核.....	47
6.1 确定模型机的功能.....	47
6.2 硬布线实现的 CU 框图及详解.....	47
6.2.1 CU 结构图.....	47
6.2.2 RAM.....	48
6.3 设计总体结构与数据通路.....	48
6.4 硬布线实现节拍发生器设计.....	51
6.5 操作码 OP 译码器的设计.....	53
6.6 指令控制信号时序及逻辑设计.....	54
6.7 指令执行流程.....	57
6.8 分部件结构.....	59
6.9 完整电路结构.....	73
七、课程设计总结.....	73

一、简单的模型机设计概述

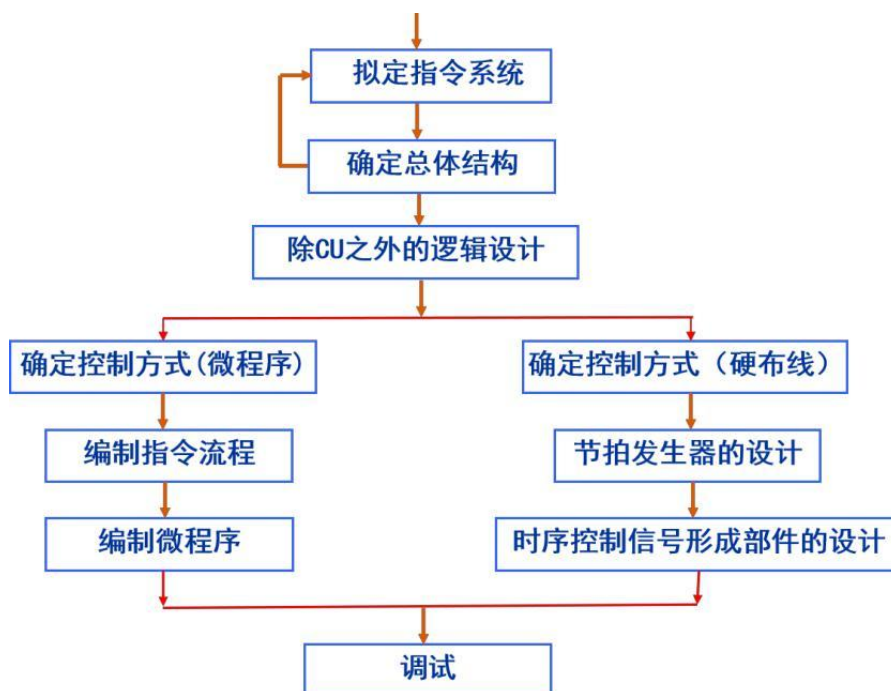
1.1 设计目的

运用计算机原理所学知识，理论联系实际，设计一台模型机：

- (1) 确定数据通路
- (2) 确定机器指令的格式，并定义相应的指令系统
- (3) 分别采用微程序与硬布线方式
- (4) 巩固课堂知识，深化学习内容

1.2 设计步骤

完整设计流程一共分为五个步骤，分别是指令系统、总体结构和数据通路、逻辑设计、CU 设计、调试。其中 CU 设计有两种方法，分别是微程序与硬布线，具体流程图如图 1 所示。



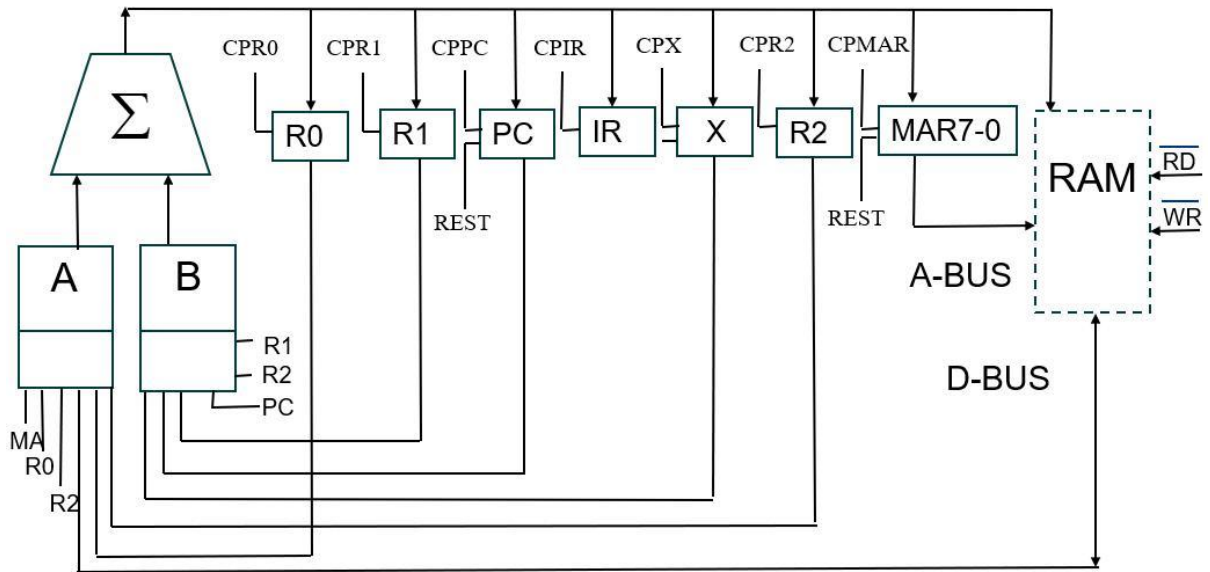
1.3 拟定指令系统

指令系统将涉及到指令字长、指令格式、指令的编码、指令种类、寻址方式。需要结合指令的多少和指令的功能来确定指令的字长、格式、种类等等。指令字长需要根据存储器的容量来确定。指令又有单字长和双字长之分。同时指令也应

该支持不同的寻址方式，比如直接寻址、间接寻址、寄存器寻址等。

1.4 确定总体结构

总体结构设计包含确定各部件设置以及它们之间的数据通路结构。数据通路不同，执行指令所需要的操作就不同，计算机的结构也就不一样。本实验采用的总体结构如下图：



1.5 逻辑设计

总体结构确定之后，便开始总体结构中各部件的逻辑设计和部件之间的连接。需要设计的器件有：R0、R1、PC、IR、X、R2、MAR、数据选择器 A、数据选择器 B 和 ALU。

1.6 确定控制方式

1.6.1 微程序实现控制器

(1) 确定微程序控制方式：

需要确定微程序控制器的结构、下地址形成方式、微程序控制器的时序、微指令格式。其中微指令格式包括微指令字段定义、微命令形成逻辑、后继微地址产生逻辑。

(2) 编制指令执行流程：

根据指令的复杂程度，确定每条指令所需要的机器周期数，对于微程序控制的计算机，根据总线结构，需考虑哪些微操作可以安排在同一条微指令。

(3) 确定微程序地址：

根据后续微地址的形成方法，确定每个微程序地址及分支转移地址。

(4) 微指令代码化：

根据微指令格式，将微程序流程中的所有微指令代码化，转化成相应的二进制代码，写入到控制存储器中的相应单元中。

(5) 调试：

用单步微指令方式执行机器指令的微程序流程图。

1.6.2 硬布线实现控制器

(1) 所有指令执行步骤划分和功能确定。

(2) 节拍发生器的设计。

(3) 根据指令执行流程和数据通路，设计各控制信号的列表。

(4) 根据列表，设计各控制信号的逻辑表达式。

(5) 设计完整电路，下载。

(6) 调试。

二、课程设计环境

2.1 硬件环境

课程设计整体基于 JYS 开放式计算机组成原理与结构实验系统开发箱，使用现场可编程逻辑阵列（FPGA），通过 EDA 设计技术，设计和下载电路实现数字逻辑实验电路。具体硬件配置如下：

(1) FPGA 电路：超大规模集成电路 FPGA 芯片 CycloneIVEEP4CE6E228C

(2) 时钟：多频率连续时钟发生器和单脉冲信号发生器

(3) RAM 存储器：用于存放指令和数据

(4) RAM 存储器：用于存放指令和数据

(5) 发光二极管指示灯及开关 5

(6) 复位信号输入：单片机复位和 CPU 复位

2.2 软件环境

本课程设计使用计算机操作系统为 Windows10，具体开发环境如下。

2.2.1 Quartus II

Altera 公司的 Quartusf 软件提供了可编程片上系统（SOPC）设计的一个综合开发环境，是进行 SOPC 设计的基础，包括系统级设计、嵌入式软件开发、编程逻辑器件设计、综合、布局和布线、验证和仿真等。

2.2.2 计算机组成原理与系统结构

负责与硬件实验平台电路调试、RAM 及 ROM 写入等。

三、指令组

3.1 微程序实验指令组

3.1.1 基本字长

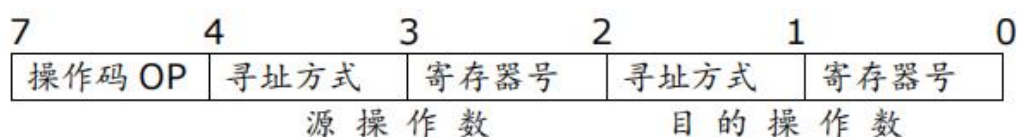
我们实验平台的存储器容量为 256×8 ，所以我们将基本字长定为 8 位。指令格式分为单字长指令和双字长指令两种。在双字长格式中，将第二字节定义为操作数或操作数地址。

3.1.2 指令格式

本模型机的指令采用统一的 8 位二进制数表示，共分为两类：

（1）单字长指令：

这类指令占用一个存储单元，仅依靠 8 位二进制数即可实现完整的功能，常见的指令为运算指令，主要有 ADD, SUB 等。



(2) 双字长指令：

这类指令需要占用两个存储单元，一般是携带操作数的指令，第一个存储单元中是指令的二进制代码，紧接着的一个内存单元中保存这条指令需要的操作数。常见的这类指令如：MOV 等。示意图如下：



本实验，0~3 号位置其实只是为了显示给用户看寻址方式和寄存器号，实质上一条指令真正如何执行是通过微操作序列结束的，与这 4 位无关，因此修改后操作码 OP 共 5 位，最多可定义 32 条指令（如下图所示）。



3.1.3 指令类型

模型机有单操作数指令、双操作数指令和无操作数指令。操作码 OP 共 4 位，最多可定义 16 条指令。由于我们拓展了一位操作码，所以本实验操作码 OP 共 5 位，最多可定义 32 条指令。下为指令类型以及相关功能解释：

(1) 运算指令: 该类指令主要涉及到运算操作，其中运算操作有加、减、乘、除、自增、自减、左移、右移等等。

指令种类	解释
ADD R0, R1	R0R1 相加送 R1
SUB R0, R1	R0R1 相减送 R1
MUL R0, R1	R0R1 相乘送 R1
MUL R0, R1, (N#)	R0R1 相乘送 RAM
DIV R1, R0, (N#)	R1R0 相除送 RAM

AND R0, R1	R0 与 R1 送 R1
OR R0, R1	R0 或 R1 相送 R1
NOR R0, R1	R0 或非 R1 相送 R1
XOR R0, R1	R0 异或 R1 相送 R1
ADD_SELF R0	R0 + 1 送 R0
SUB_SELF R0	R0 - 1 送 R0
NOT_SELF R0	R0 取反送 R0
LSL R0	R0 左移 1 位送 R0
LSR R0	R0 右移 1 位送 R0
ADD_SELF X	X 自增
ADD X, (N#)	从指定地址开始累加
MUL X, (N#)	从指定地址开始累乘

(2) 访存指令:这类指令主要涉及到访存操作, 其中访存操作有立即数送 R0、立即数送 R1、R0 送 RAM、R1 送 RAM 等等。

指令种类	解释
MOV1 N#, R0	N 送入 R0
MOV2 N#, R1	N 送入 R1
MOV3 R0, (N#)	R0 送 RAM
MOV4 R1, (N#)	R1 送 RAM
MOV7 R0, ((N#))	R0 送 RAM(间接寻址)
MOV8 R1, ((N#))	R1 送 RAM(间接寻址)
MOV9 N#, X	N 送入 X

(3) 循环跳转指令:这类指令主要涉及到循环跳转操作, 主要是为实现在 RAM 中设计的三个函数, 以及进行相应的封装。

指令种类	解释
MOV5 PC, (N#)	PC 送 RAM
MOV6 RAM(N#), PC	RAM 送 PC
MOV10 PC, R2	PC 送 R2
MOV11 R2, PC	R2+1 送 PC
MOV12 (N#), PC	将 RAM (N#) 送 PC
JNZ	判断循环是否结束

(4) 停机指令：该指令会结束程序。

指令种类	解释
HALT	停机

3.1.4 地址码寻址方式

当寻址方式位为 0，是寄存器寻址，操作数在指定的寄存器中，相应的寄存器号位为 0 是 R0，为 1 是寄存器 R1。当寻址方式位为 1 时，寻址方式位和寄存器号位组合 10：是立即数寻址，操作数在指令的下一个单元；11：是直接寻址，操作数地址在指令的下一个单元。本实验中由于需要拓展指令，寻址方式不会在指令中展现出来。

3.2 硬布线实验指令组

在硬布线实验中，采用 4 位操作码，一共有 16 条指令，其中包括运算指令和访存指令。

(1) 运算指令：该类指令主要涉及到运算操作，其中运算操作有加、减、乘、除、自增、自减、左移、右移等等。

指令种类	解释
ADD R0, R1	R0R1 相加送 R0
SUB R0, R1	R0R1 相减送 R0
MUL R0, R1	R0R1 相乘送 R0
DIV R0, R1	R0R1 相除送 R0
AND R0, R1	R0 与 R1 送 R0
OR R0, R1	R0 或 R1 送 R0
NOR R0, R1	R0 或非 R1 相 R0
XOR R0, R1	R0 异或 R1 相 R0
ADD_SELF R0	R0 + 1 送 R0
SUB_SELF R0	R0 - 1 送 R0
LSL R0	R0 左移 1 位送 R0
LSR R0	R0 右移 1 位送 R0

(2) 访存指令：这类指令主要涉及到访存操作，其中访存操作有立即数送 R0、立即数送 R1、R0 送 RAM 等等。

指令种类	解释
MOV1 N#, R0	N 送入 R0
MOV2 N#, R1	N 送入 R1
MOV3 R0, (N#)	R0 送 RAM

(3) 停机指令：该指令会结束程序。

指令种类	解释
HALT	停机

四、总体结构与分部件结构

4.1 设计概述

由于组合逻辑控制器用节拍发生器（几个触发器构成的时序逻辑电路）不同的状态组合来区分一条指令不同的执行步骤，指令执行步骤的接续是通过变换节拍发生器的状态组合完成的，不同于微程序控制器中通过微地址形成部件给出不同的微指令地址来实现。组合逻辑控制器是通过由“与-或”两级逻辑关系构成的时序控制信号产生部件来直接给出全部的时序控制信号。这一部分仅展示微程序实现的分部件结构，硬布线实现的分部件结构在第六部分。

4.1.1 寻址方式

本实验中设计的微程序控制器支持多种寻址方式，如立即数寻址、直接寻址、寄存器寻址、变址寻址、间接寻址等等，其中 MOV1、MOV2 等指令为立即数寻址，即立即数跟着指令后面。MOV3、MOV4 等指令为寄存器寻址，操作数在寄存器中。MOV7、MOV8 等指令为间接寻址，而其他的 MOV 指令的设计都是为了方便实现的三个函数以及进行相应的封装，在函数中会涉及到变址寻址等寻址方式。

4.1.2 设计简述

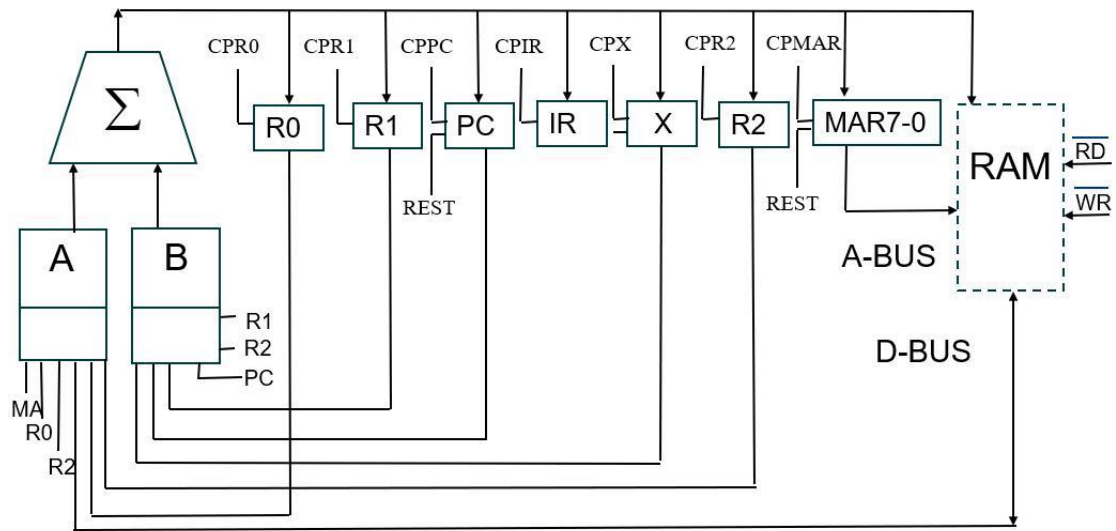
(1) 设计一个八位算术逻辑运算单元 ALU。

(2) 熟悉微命令的产生和时序。

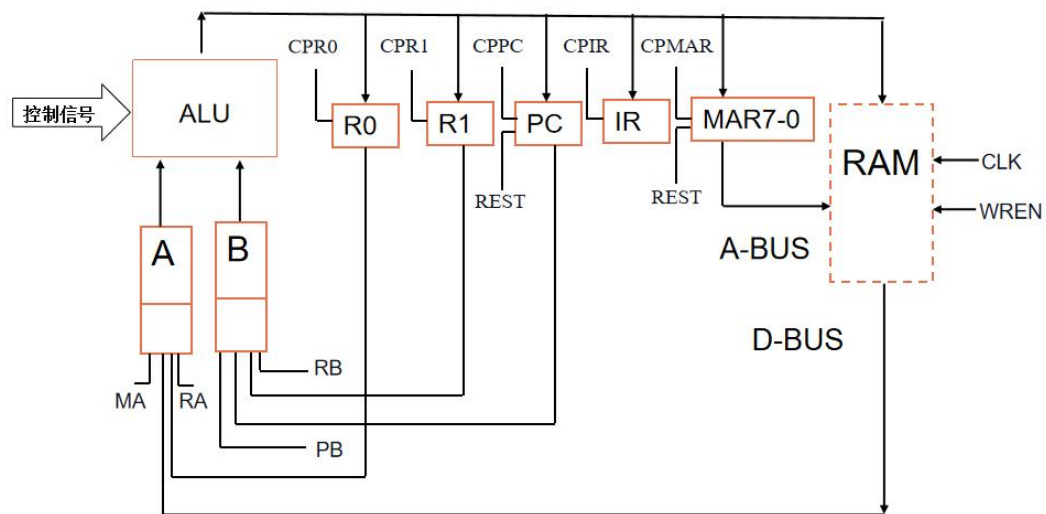
(3) 两操作数由两个八位寄存器 R0、R1 提供，其结果放入 R1 等寄存器中。具体执行的操作可由微命令任意设定。

4.2 总体结构框图

4.2.1 微程序设计总体框图



4.2.2 硬布线设计总体框图



4.3 寄存器

(1) 不带复位的寄存器

结构中 R_0 、 R_1 、 R_2 通用寄存器，可存放操作数或结果、中间结果，每个寄存器均由 8 个 D 触发器构成。在 CPR_i 的作用下接收总线的数据送入寄存器，输出连入选择器。指令寄存器 IR 和变址寄存器 X 结构同通用寄存器。

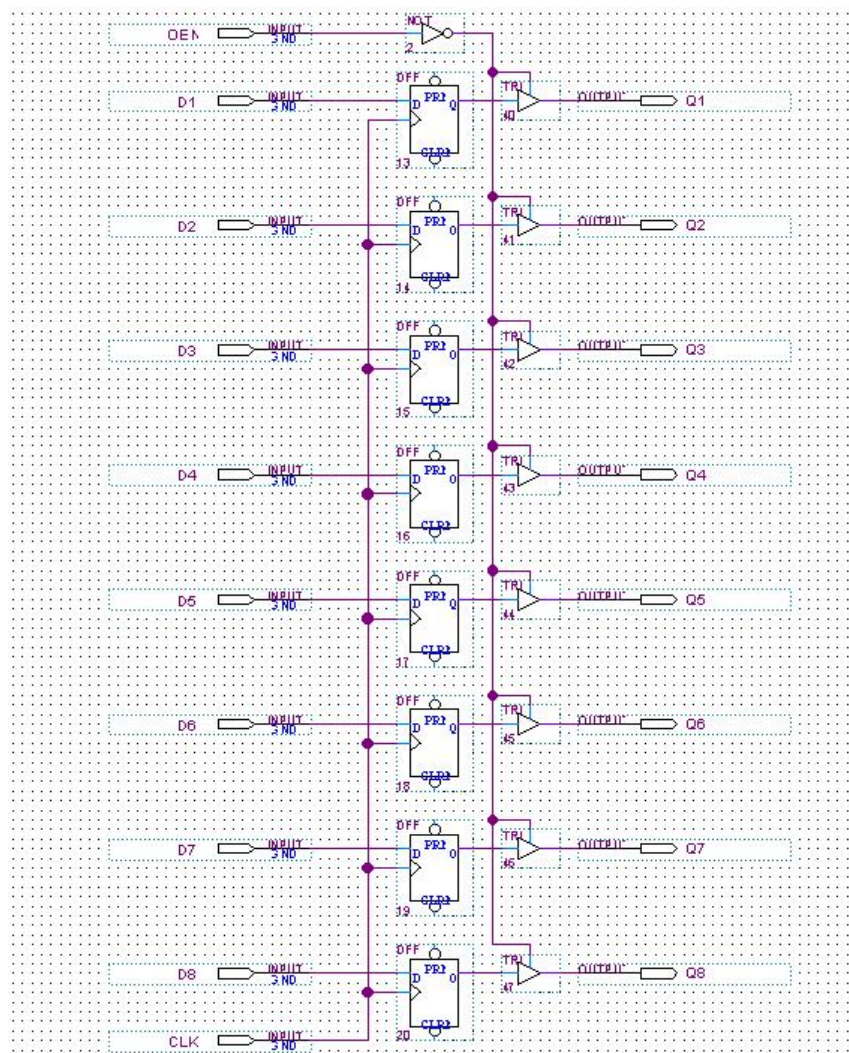
(2) 带复位的寄存器

结构中 MAR 地址寄存器是一个带复位的寄存器，带复位是指当有复位信号时，MAR 清零。

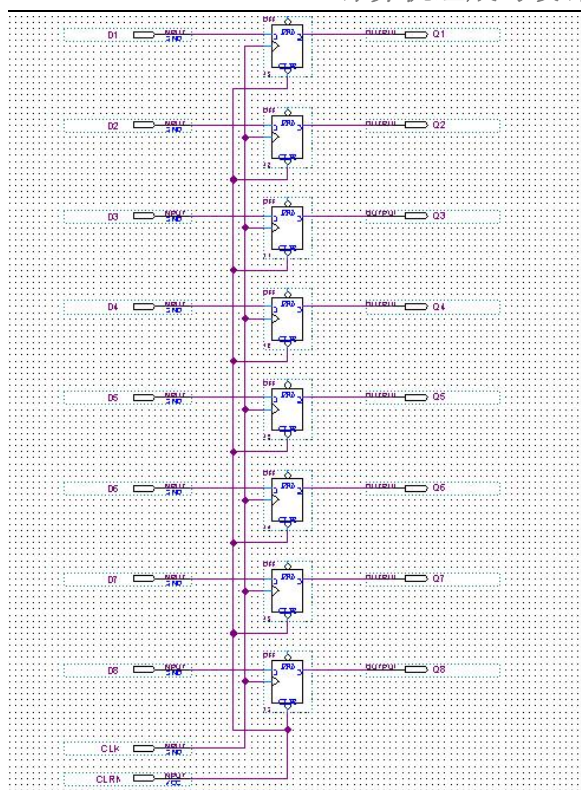
(3) 程序计数器 PC 的设计

程序计数器是有复位信号的 8 位寄存器。PC 加 1 是通过加法器实现的。复位信号的作用是有复位信号时，计数器 PC 清零。

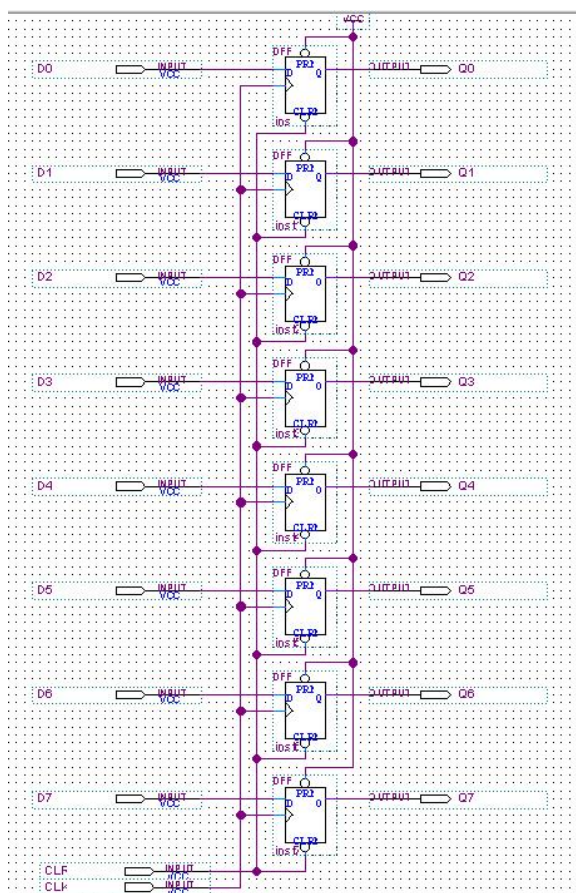
R0、R1、R2、X 寄存器



MAR 寄存器：



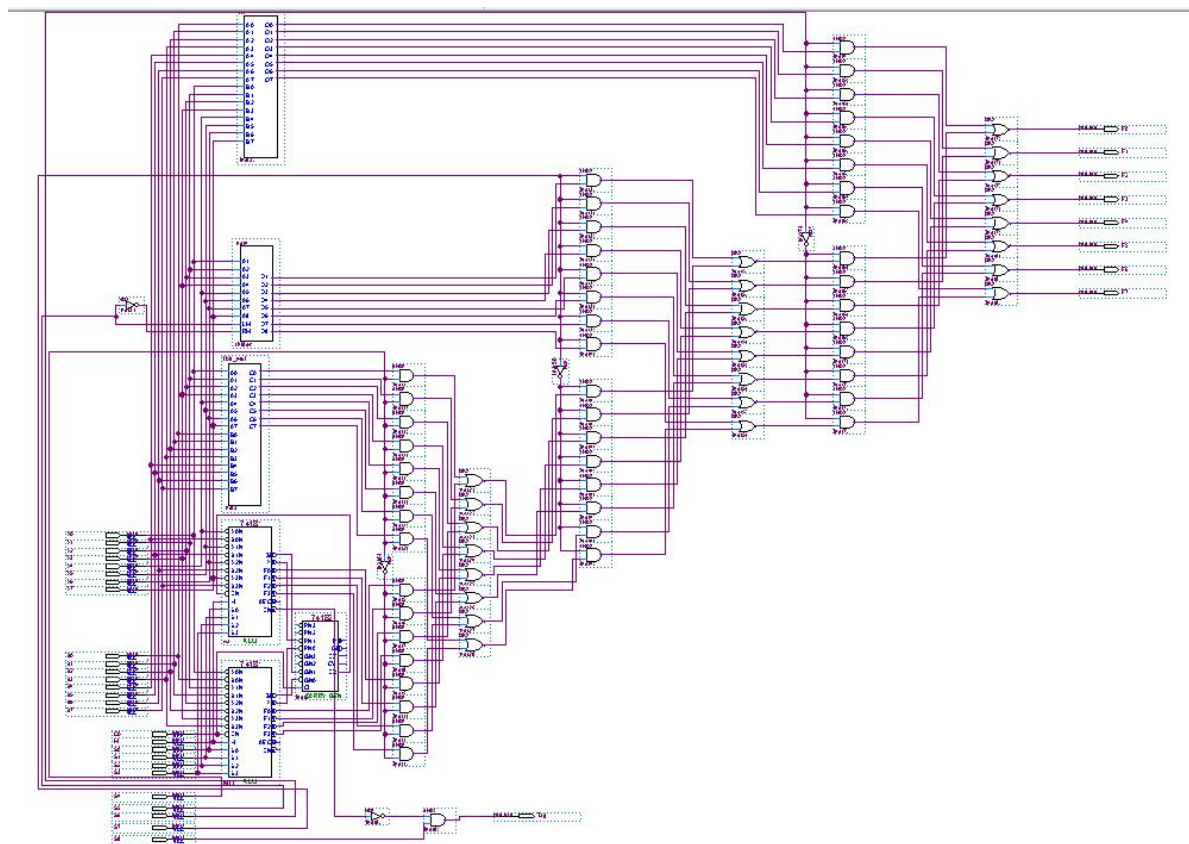
PC 寄存器:



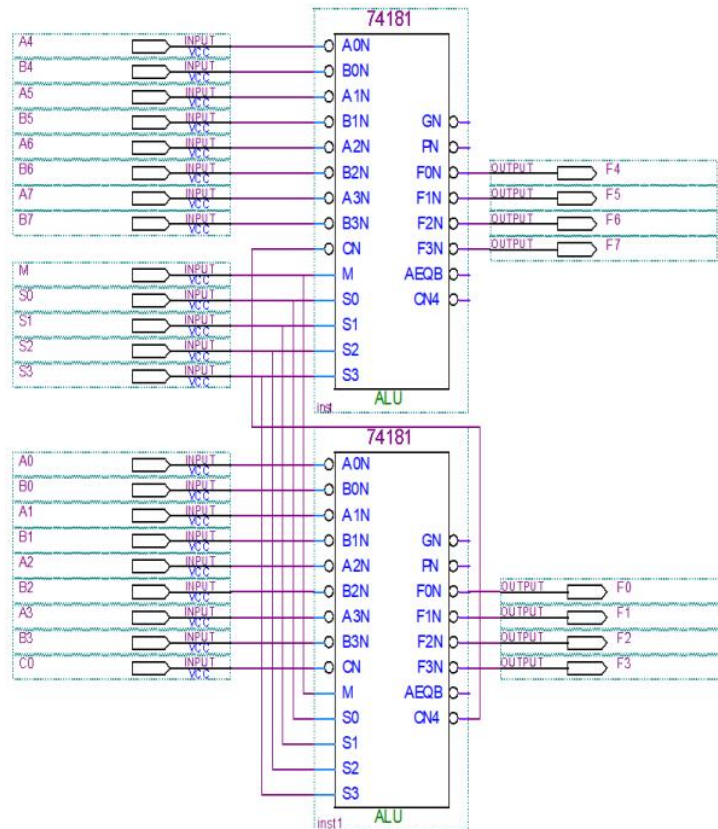
4.4 运算器

本模型机的运算器实现是基于 74181 加法器的 ALU, 并使用了 74182 实现了超前进位并行加法器。在此基础上扩展了乘法、除法、移位等运算, 处理字长为 8 位:

ALU 整体电路:



(1) 基本加减逻辑运算器件



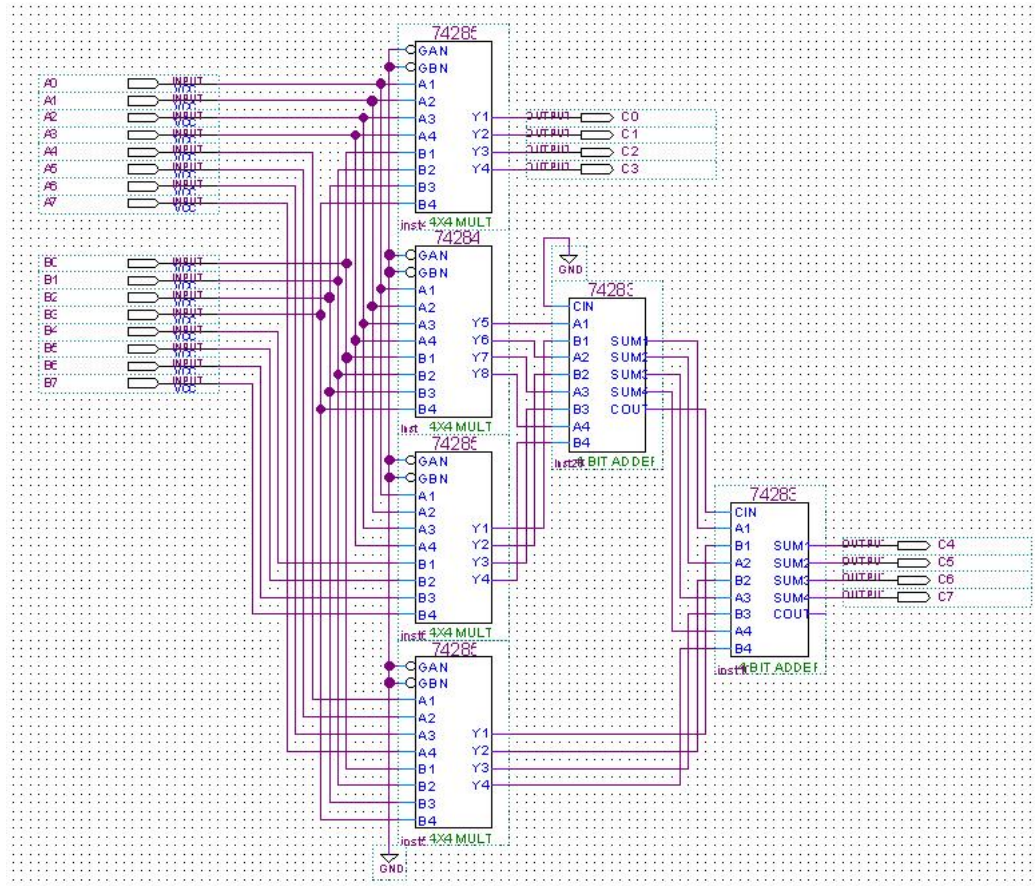
借助 74181 芯片我们可以实现除加法外功能，通过 M，CN 和 S0、S1、S2、S3 来控制运算，加法器的部分功能如下图。

方式	M=1 逻辑运算	M=0 算术运算	
S3 S2 S1 S0	逻辑运算	CN=1 (无进位)	CN=0 (有进位)
0 0 0 0	$F=A$	$F=A$	$F=A$ 加 1
0 0 0 1	$F=(A+B)$	$F=A+B$	$F=(A+B)$ 加 1
0 0 1 0	$F=(A)B$	$F=A+B$	$F=(A+B)$ 加 1
0 0 1 1	$F=0$	$F=$ 负 1	$F=0$
0 1 0 0	$F=(AB)$	$F=A$ 加 $A(B)$	$F=A$ 加 A/B 加 1
0 1 0 1	$F=B$	$F=(A+B)$ 加 A/B	$F=(A+B)$ 加 A/B 加 1
0 1 1 0	$F=A \oplus B$	$F=A$ 减 B 减 1	$F=A$ 减 B
0 1 1 1	$F=A/B$	$F=A(B)$ 减 1	$F=A(B)$
1 0 0 0	$F=A+B$	$F=A$ 加 AB	$F=A$ 加 AB 加 1
1 0 0 1	$F=(A \oplus B)$	$F=A$ 加 B	$F=A$ 加 B 加 1
1 0 1 0	$F=B$	$F=(A+B)$ 加 AB	$F=(A+B)$ 加 AB 加 1
1 0 1 1	$F=AB$	$F=AB$ 减 1	$F=AB$
1 1 0 0	$F=1$	$F=A$ 加 A	$F=A$ 加 A 加 1
1 1 0 1	$F=A+B$	$F=(A+B)$ 加 A	$F=(A+B)$ 加 A 加 1
1 1 1 0	$F=A+B$	$F=(A+B)$ 加 A	$F=(A+B)$ 加 A 加 1
1 1 1 1	$F=A$	$F=A$ 减 1	$F=A$

(上表中的 “/” 表示求反)

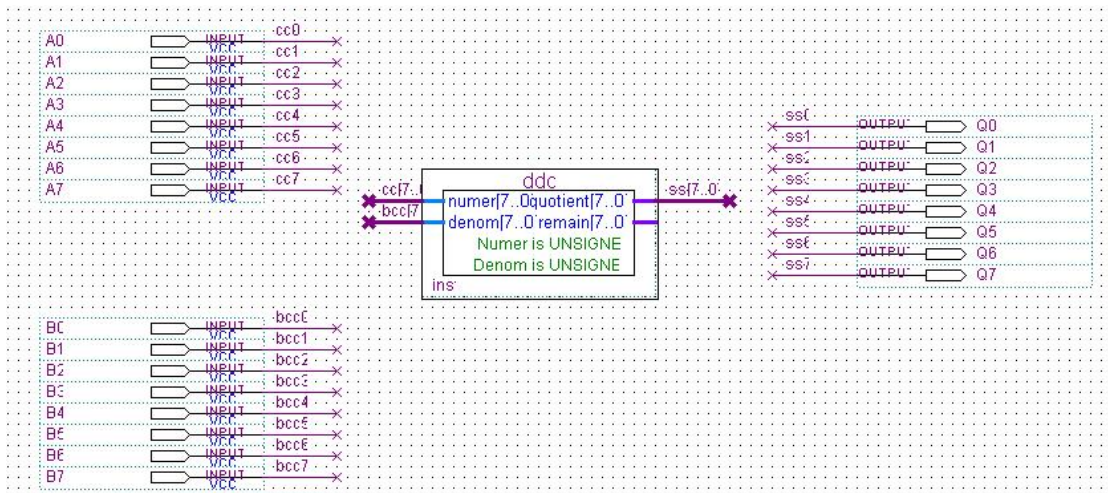
(2) 乘法器

通过组合 74285 芯片和 74284 芯片来实现乘法功能，也可以通过设计阵列乘法器来完成。

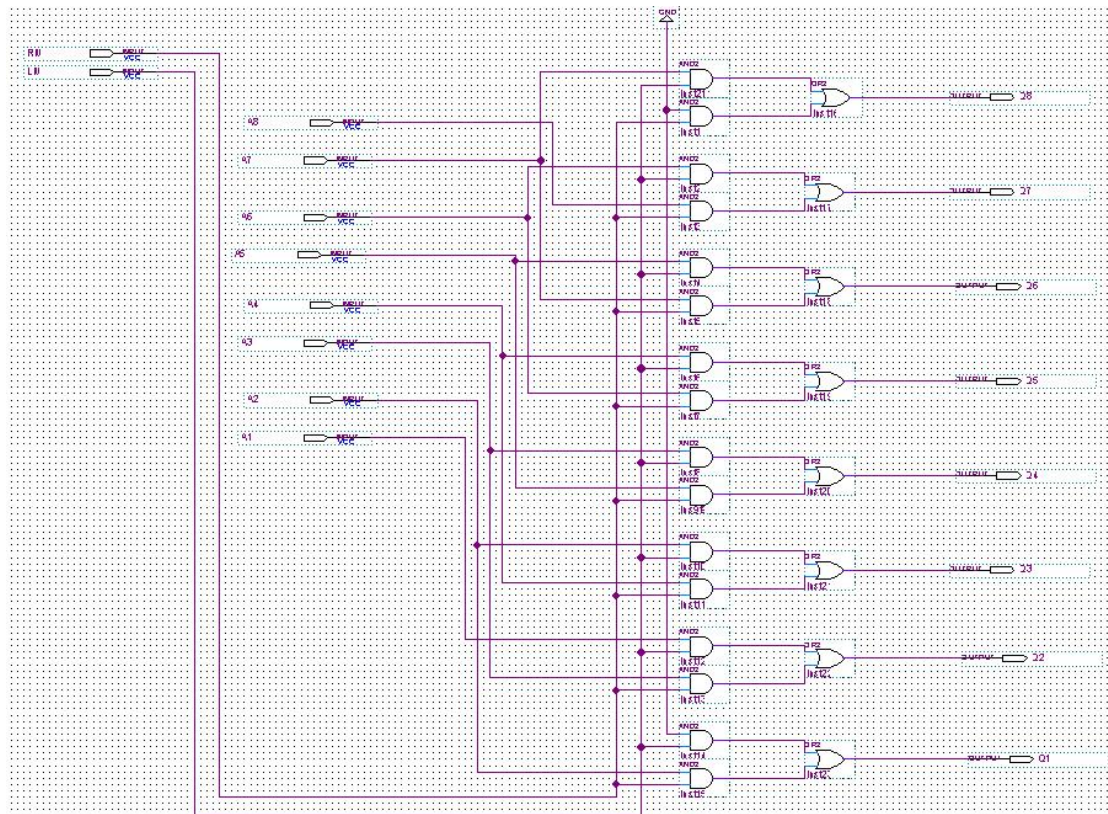


(3) 除法器

使用 Quartus | 软件中自带的除法器来完成。



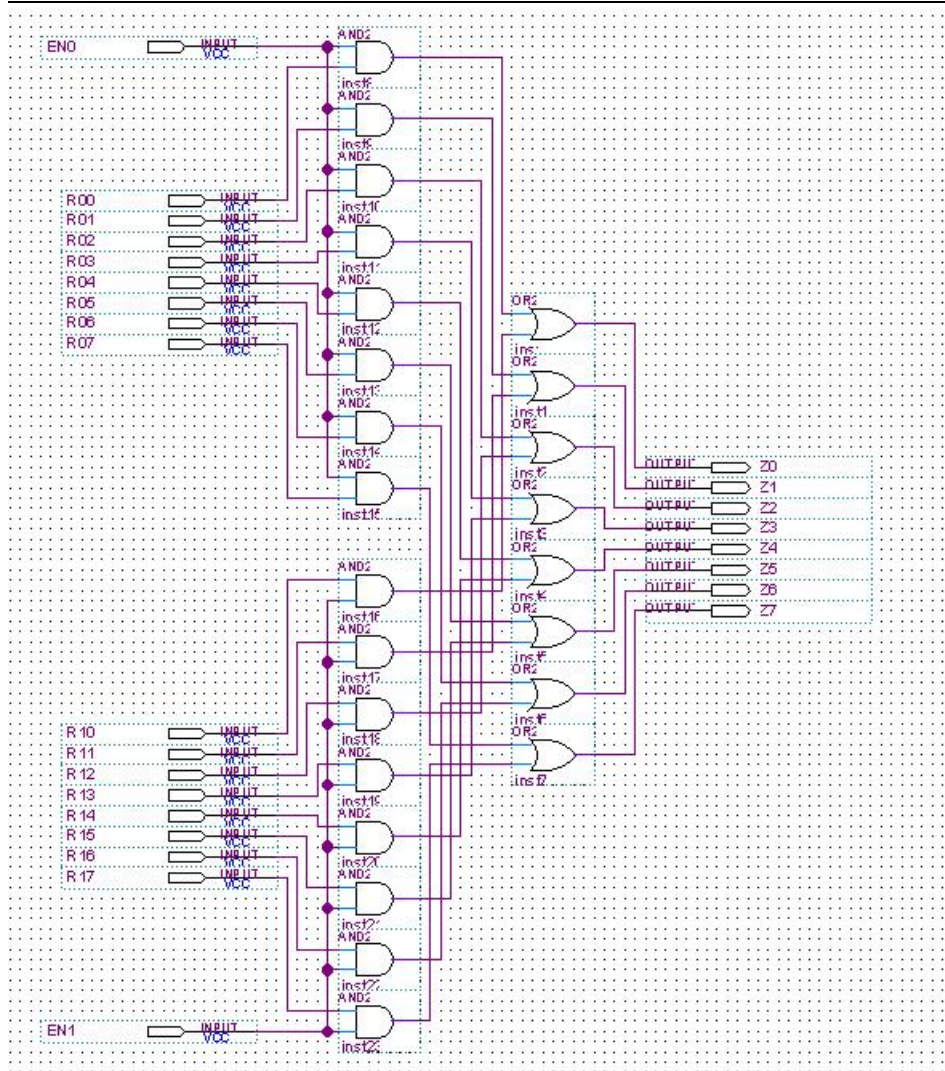
(4) 移位器



4.5 选择器

本实验中有 2 选 1 数据选择器和 3 选 1 数据选择器两种数据选择器，其中 2 选 1 数据选择器是用于下地址生成部分，而 3 选 1 数据选择器用于 ALU 输入端。

2 选 1 数据选择器：

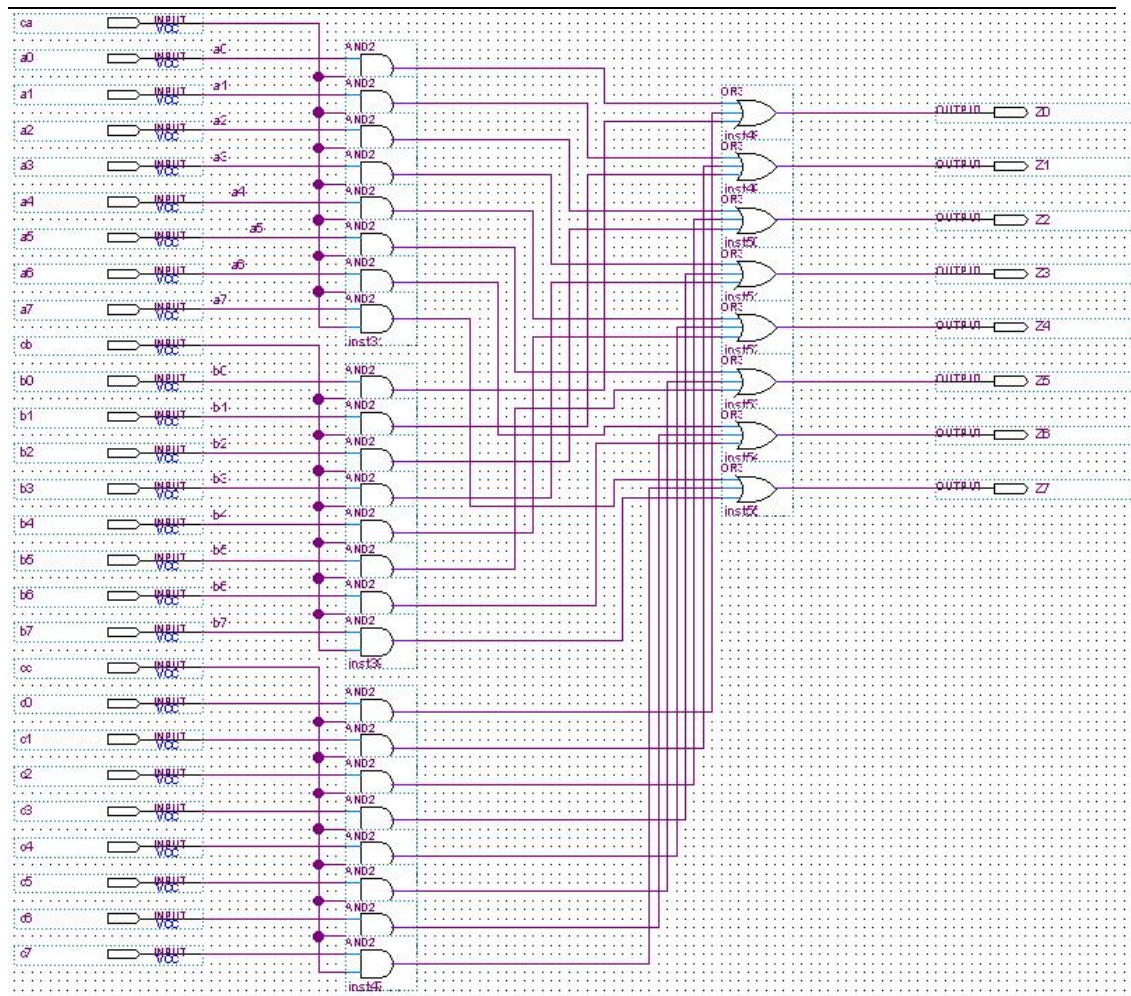


ALU 的两个输入端分别接入两个 3 选 1 数据选择器：

(1) 连入 A 选择器的数据来源于 RAM 的读出数据和 R0 寄存器和 R2 寄存器的数据。

(2) 连入 B 选择器的数据来源于 PC 的数据和 R1 寄存器和 X 寄存器的数据。

3 选 1 数据选择器：



4.6 UPC 设计

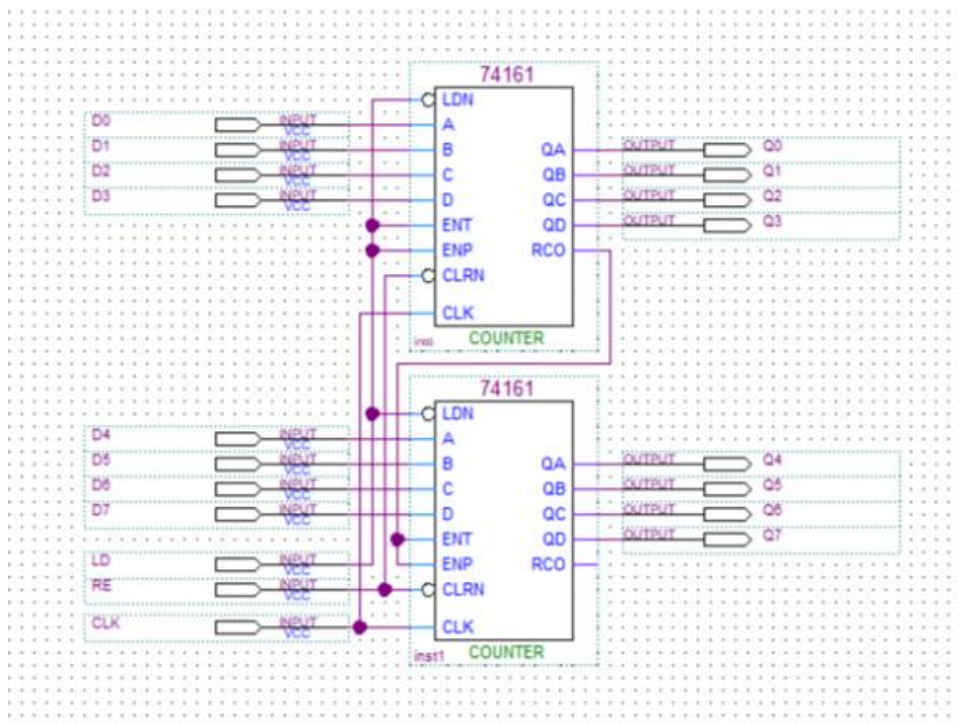
CLR : 清零端，低电平有效；

CLR=0 时，Q7Q6Q5Q4Q3Q2Q1Q0=00000000；

LOAD: 置数端，低电平有效；

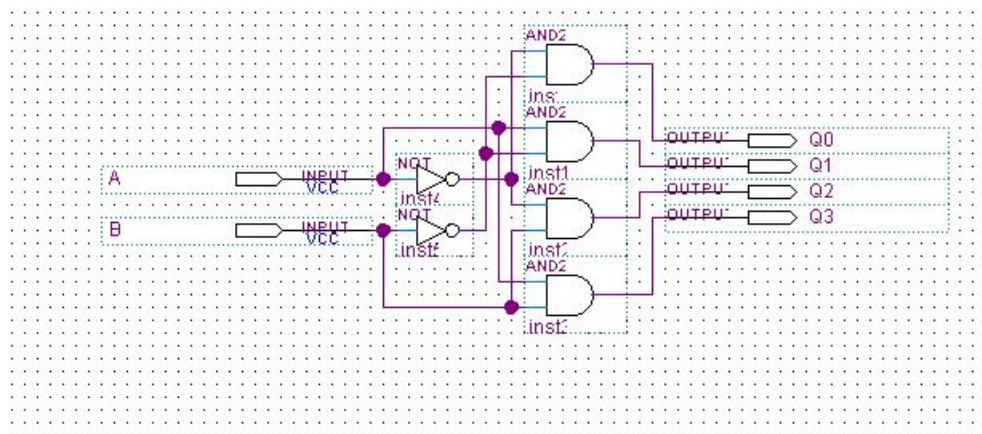
LOAD=0 时，在 CLK 的上升沿，Q7Q6Q5Q4Q3Q2Q1Q0=D7D6D5D4D3D2D1D0；

当 CLR=1，LOAD=1，ET=1，EP=1 时，对 CLK 进行增 1 计数。

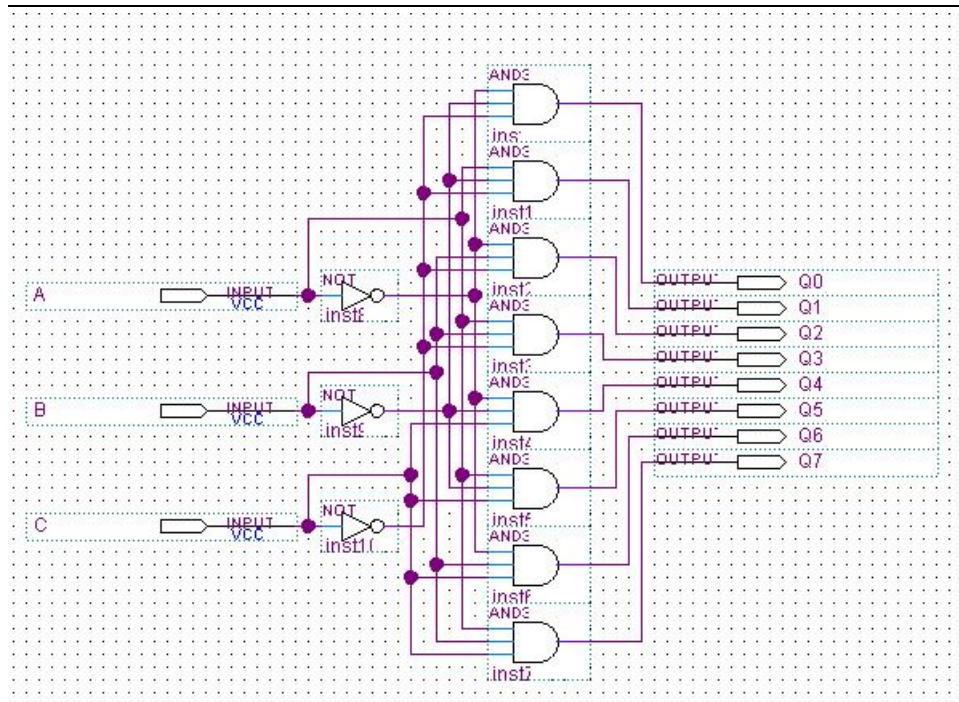


4.7 译码器

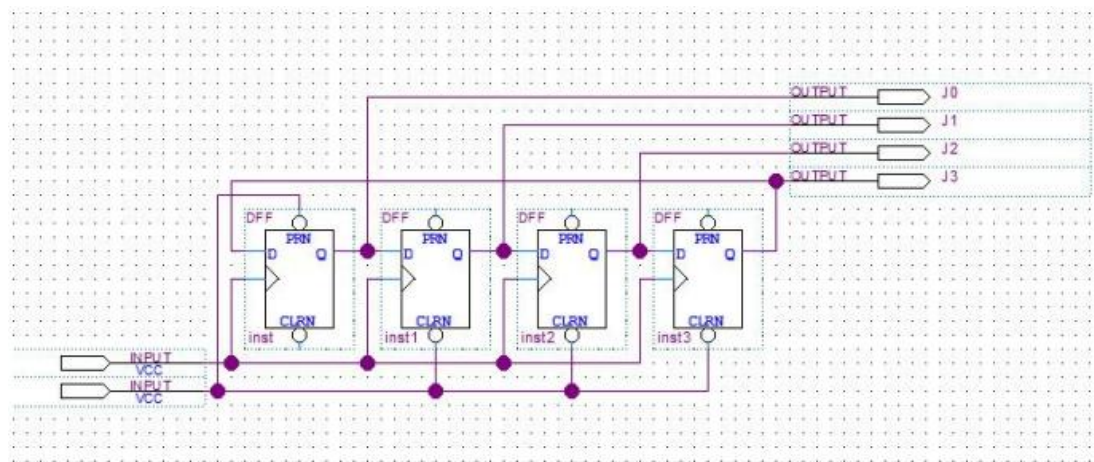
(1) 2-4 译码器



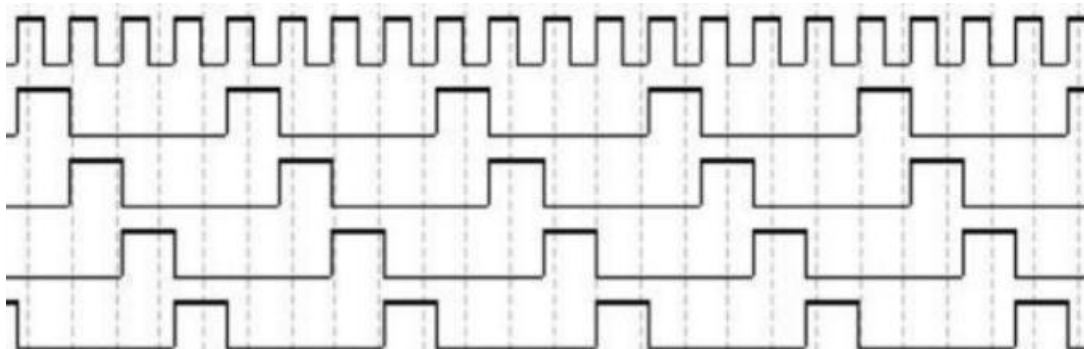
(2) 3-8 译码器



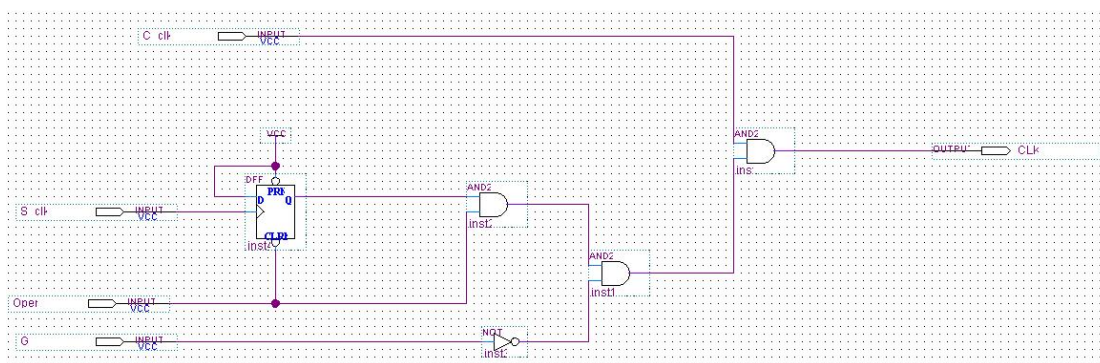
4.8 D 触发器四节拍发生器



产生节拍示意图如下：



4.9 开关装置（启动电路）



五、微程序实现的模型机内核

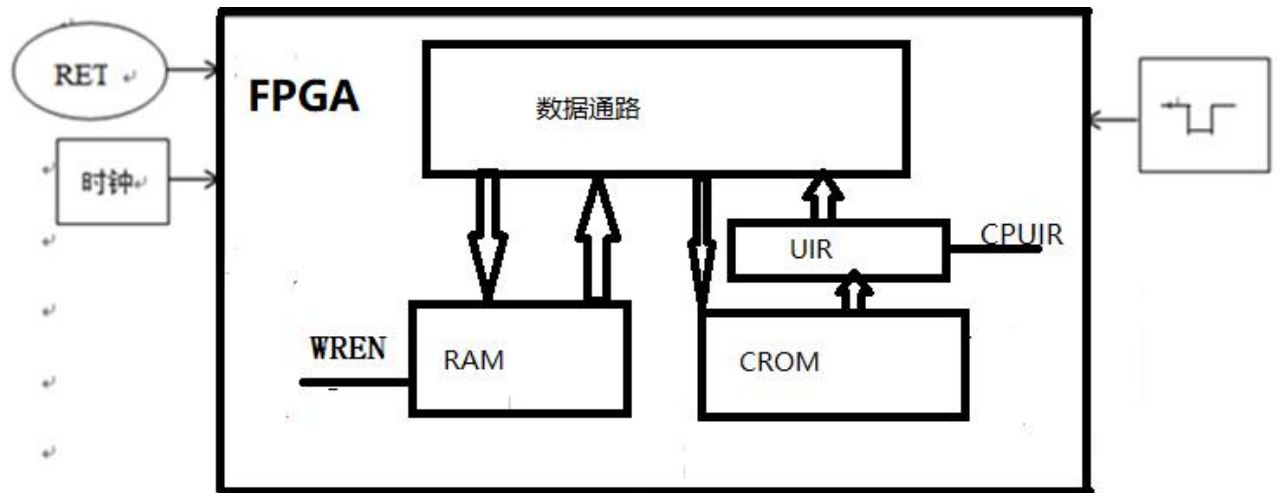
5.1 确定模型机的功能

(1)微程序模型机应实现以下功能：加、减、乘、除、与、或、非、异或、或非、自增、自减、逻辑左移、逻辑右移。

(2)同时微程序模型机应支持多种寻址方式：立即数寻址、寄存器寻址、间接寻址、寄存器间接寻址、变址寻址等。

(3)模型机的特色功能：实现了三个函数，其中先是利用变址寻址实现 do while 循环，然后通过循环实现了累加、累乘两个函数，之后还设计了求幂的函数，不过限于 ALU 只能进行八位运算，所以无法运算更复杂的求幂操作，通过相关的 MOV 指令，对函数进行相应的封装，以便更好地调用。

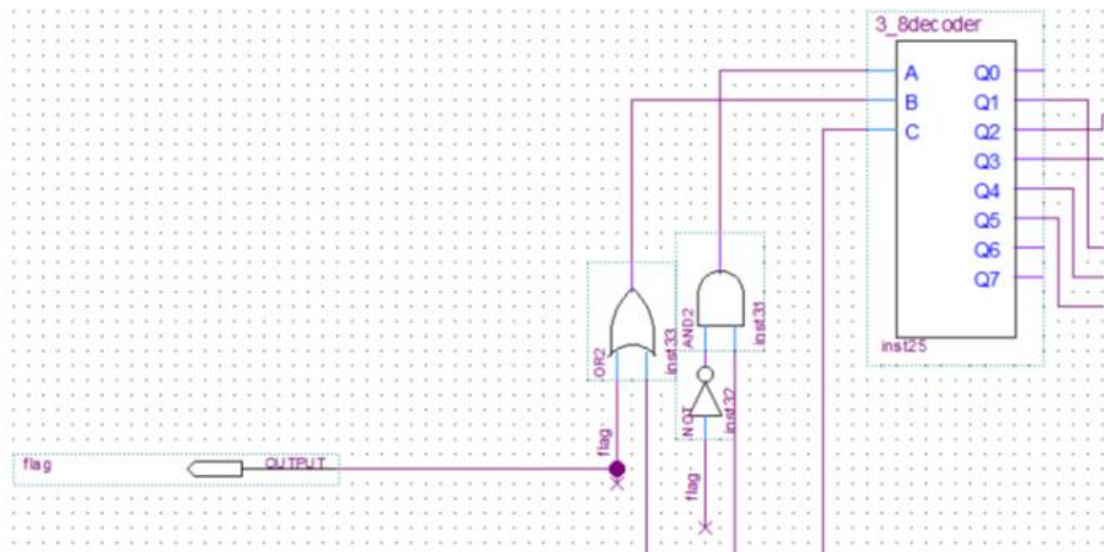
5.2 微程序实现的 CU 框图及详解



微程序控制器的工作流程为：

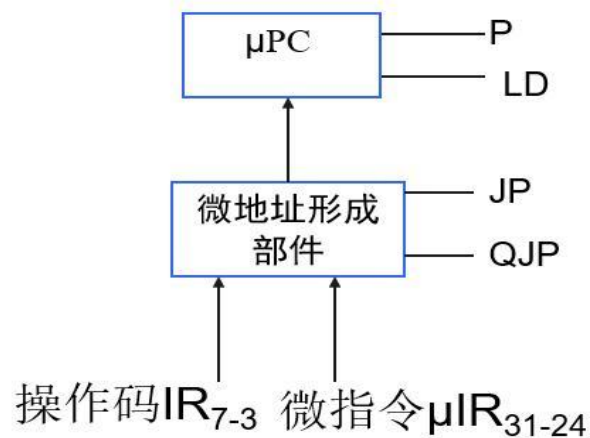
- (1) 后继地址形成电路根据当前指令（从 RAM 中读到）生成一条微程序地址送给 upc。
- (2) upc 将地址送给微程序存储器 ROM 的 address 端。
- (3) 微程序存储器 ROM 根据地址读出一条微指令。
- (4) 微指令解析电路将微指令解释成各个控制位，发给总体结构，按照设计的数据通路执行这条指令。
- (5) 由每组微指令的最后一条指令（JP 或 QJP）决定 upc 的下一个跳转位置。

5.2.1 下地址形成电路



(1) 后继微地址形成电路

后继微地址形成电路负责形成微程序控制中下一条微指令的地址，实现了三种方式，分别为顺序执行、JP 无条件转移和 QJP 按操作码转移，框图如下：



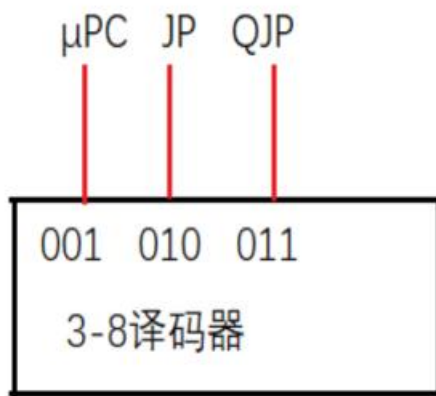
当 $LD=1$ 时，微程序计数 μPC 执行加 1 操作。

当 $LD=0$ 时且 $JP=1$ 时，无条件转移，由微指令的中八位提供转移地址。

当 $LD=0$ 时且 $QJP=1$ 时，按操作码转移。

(2) 后继地址形成电路

使用 3-8 译码器生成信号，微地址形成部件使用 2 选 1 选择器来实现。



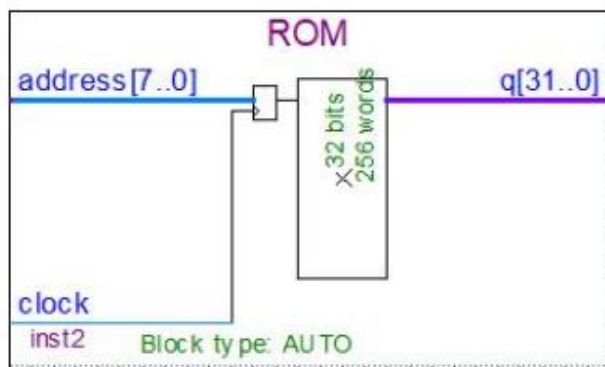
当 JP=1, QJP=0 时, 数据选择器输出 = $\mu IR_{31} \mu IR_{30} \mu IR_{29} \mu IR_{28} \mu IR_{27} \mu IR_{26} \mu IR_{25} \mu IR_{24}$

当 JP=0, QJP=1 时, 数据选择器输出 = $IR_7 IR_6 IR_5 IR_4 IR_3 000$

数据选择器输出连接 μPC 的输入即可。

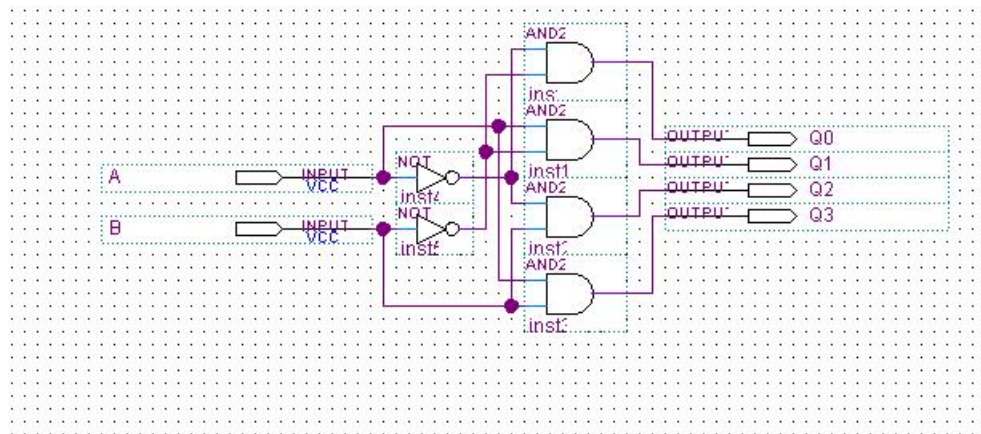
5.2.2 控制存储器 ROM

控制存储器即 ROM, 用于存储微程序, 我们对 ROM 进行了拓展, 将 ROM 由原来 24 位拓展到 32 位, 并将高八位作为跳转指令的跳转地址。

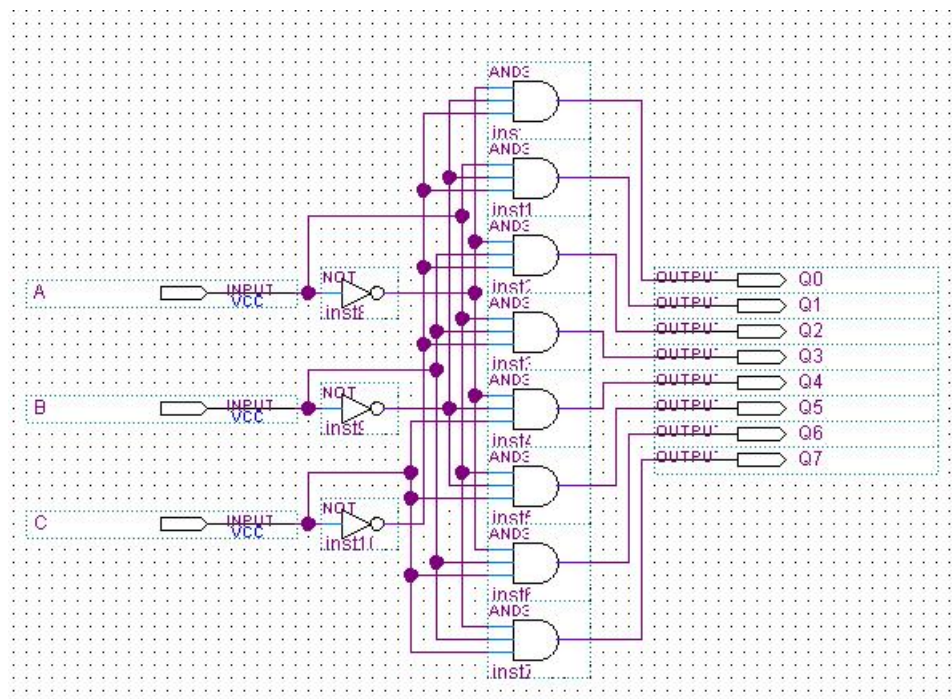


5.2.3 译码器

(1) 2-4 译码器

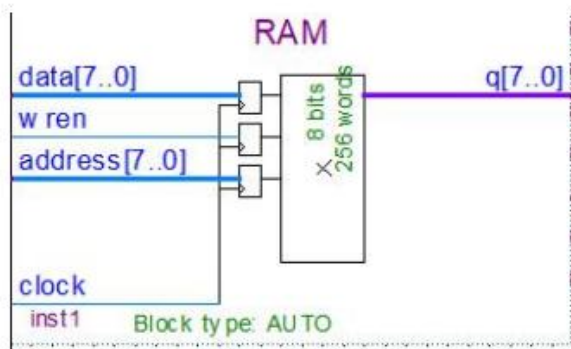


(2) 3-8 译码器



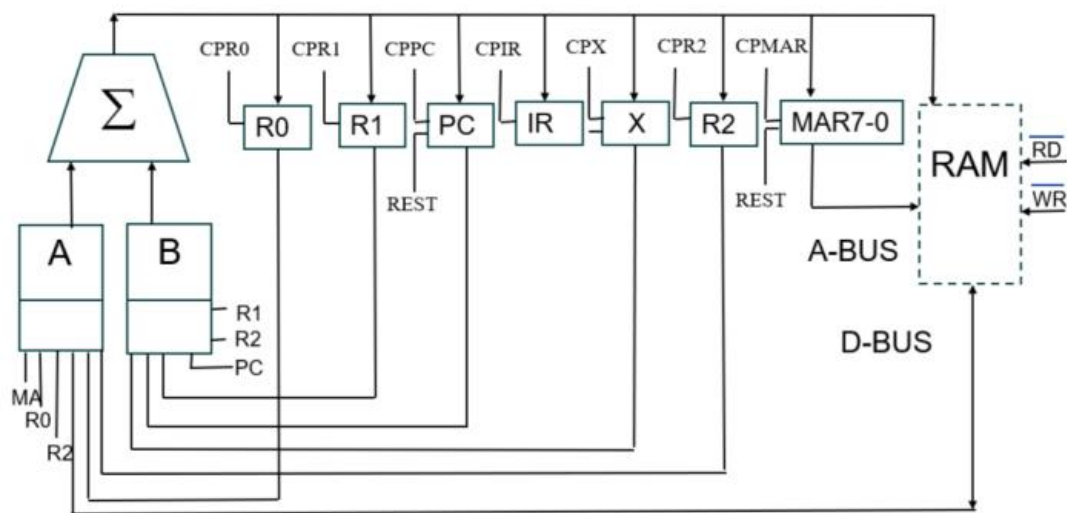
5.2.4 RAM

用于存放数据和地址。其存放的地址，一种是经过下地址形成电路，形成取下一条微指令的地址。另一种是指明执行指令的数据要存放在 RAM 的地址，而数据则是直接跟在指令后。



5.3 设计总体结构与数据通路

(1) 总体结构示意图



(2) 寄存器组的设置

R_0 、 R_1 、 R_2 为通用寄存器，8 位；

X 为变址寄存器，8 位；

IR 为指令寄存器，8 位；

PC 程序计数器，8 位；

MAR 为地址寄存器，8 位。

(3) ALU 的设置

采用 74181、74182 实现以下功能：

方式	M=1 逻辑运算	M=0 算术运算	
S3 S2 S1 S0	逻辑运算	CN=1 (无进位)	CN=0 (有进位)
0 0 0 0	$F=A$	$F=A$	$F=A$ 加 1
0 0 0 1	$F=A+B$	$F=A+B$	$F=(A+B)$ 加 1
0 0 1 0	$F=A/B$	$F=A+B$	$F=(A+B)$ 加 1
0 0 1 1	$F=0$	$F=$ 负 1	$F=0$
0 1 0 0	$F=A \oplus B$	$F=A$ 加 A (/B)	$F=A$ 加 A/B 加 1
0 1 0 1	$F=B$	$F=(A+B)$ 加 A/B	$F=(A+B)$ 加 A/B 加 1
0 1 1 0	$F=A \oplus B$	$F=A$ 减 B 减 1	$F=A$ 减 B
0 1 1 1	$F=A/B$	$F=A (/B)$ 减 1	$F=A (/B)$
1 0 0 0	$F=A+B$	$F=A$ 加 AB	$F=A$ 加 AB 加 1
1 0 0 1	$F=A \oplus B$	$F=A$ 加 B	$F=A$ 加 B 加 1
1 0 1 0	$F=B$	$F=(A+B)$ 加 AB	$F=(A+B)$ 加 AB 加 1
1 0 1 1	$F=AB$	$F=AB$ 减 1	$F=AB$
1 1 0 0	$F=1$	$F=A$ 加 A	$F=A$ 加 A 加 1
1 1 0 1	$F=A+B$	$F=(A+B)$ 加 A	$F=(A+B)$ 加 A 加 1
1 1 1 0	$F=A+B$	$F=(A+B)$ 加 A	$F=(A+B)$ 加 A 加 1
1 1 1 1	$F=A$	$F=A$ 减 1	$F=A$

(上表中的 “/” 表示求反)

采用 74283、74284、74285，实现乘法功能；

采用基本与或门，实现移位功能；

采用系统调用，实现除法功能；

(4) 选择器的设置

连入 A 选择器的数据来源是 RAM 的读出数据和 R0 寄存器和 R2 寄存器的数据。

连入 B 选择器的数据来源是 PC 的数据和 R1 寄存器和 X 寄存器的数据。

(5) 数据通路

模型机的数据通路是以总线为基础, 以 CPU 为核心构成的。

① 取指令

$$\text{RAM} \xrightarrow{\text{RAM}} \text{选择器 A} \xrightarrow{\text{A 直传}} \sum \rightarrow \text{Bus} \xrightarrow{\text{CPIR}} \text{IR}$$

② 送指令地址

$$\text{PC} \xrightarrow{\text{PC}} \text{选择器 B} \xrightarrow{\text{B 直传}} \sum \rightarrow \text{Bus} \xrightarrow{\text{CPMAR}} \text{MAR}$$

③ 指令计数器+1

$$\text{PC} \xrightarrow{\text{PC}} \text{选择器 B} \xrightarrow{\text{A+B+1}} \sum \rightarrow \text{Bus} \xrightarrow{\text{CPPC}} \text{PC}$$

④ RAM→R (R 指任意一个寄存器)

$$\text{RAM} \xrightarrow{\text{RAM}} \text{选择器 A/B} \xrightarrow{\text{A/B 直传}} \sum \rightarrow \text{Bus} \xrightarrow{\text{CPR}} \text{R}$$

⑤ R→RAM (R 指任意一个寄存器)

$$\text{R} \xrightarrow{\text{R}} \text{选择器 A/B} \xrightarrow{\text{A/B 直传}} \sum \rightarrow \text{Bus} \xrightarrow{\text{WR}} \text{RAM}$$

⑥ RA*RB→RC (RA 指选择器 A 控制的任意寄存器, RB 指选择器 B 控制的任意寄存器, C 指任意寄存器, *表示某种运算)

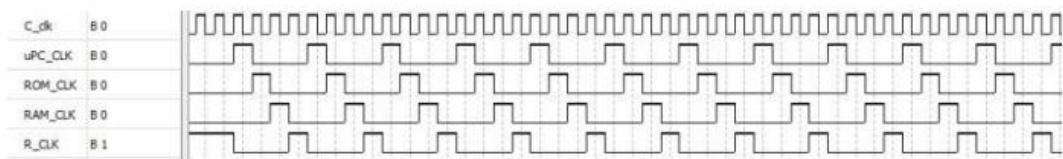
$$\text{RA、RB} \xrightarrow{\text{RA、RB}} \text{选择器 A、B} \xrightarrow{\text{运算}} \sum \rightarrow \text{Bus} \xrightarrow{\text{RC}} \text{RC}$$

⑦ RA*RB→RAM (RA 指选择器 A 控制的任意寄存器, RB 指选择器 B 控制的任意寄存器, *表示某种运算)

$$\text{RA、RB} \xrightarrow{\text{RA、RB}} \text{选择器 A、B} \xrightarrow{\text{运算}} \sum \rightarrow \text{Bus} \xrightarrow{\text{WR}} \text{RAM}$$

5.4 微程序控制时序

采用四节拍发生器来产生时序。时钟连续脉冲作为节拍发生器的输入。第一个节拍控制 uPC 的脉冲, 第二个节拍控制 ROM 的脉冲, 第三个节拍控制 RAM 的脉冲, 第四个节拍控制寄存器组的脉冲。通过节拍发生器产生脉冲, 不会出现时序竞争问题, 比较安全。如下图:



5.5 微指令格式

微指令字段定义：微指令字长 32 位，即 $\mu IR_{31} \sim \mu IR_0$ (原本 24 位，高 8 位是为实现转跳地址进行的拓展)。

ALU 控制	$\mu IR_{21} \mu IR_{20} \mu IR_{19} \mu IR_{18} \mu IR_{17} \mu IR_{16}$ MS3S2S1S0C0 μIR_{23} : S7 控制移位 1 进行移位 0 不进行移位 μIR_{22} : S4 控制乘法 1 进行乘法 0 不进行乘法 μIR_8 : S8 转跳时比较 1 进行比较 0 不进行比较 μIR_7 : S6 控制除法 1 进行除法 0 不进行除法 μIR_6 : S5 控制左右移 1 进行左移 0 进行右移	
停机控制	μIR_3 1 停机 0 运行	
A 选择器控制	$\mu IR_{15} \mu IR_{14}$ 0 0 备用 0 1 R0 1 0 RAM 1 1 R2	2-4 译码器，互斥

B 选择器控制	$\mu IR_{13}\mu IR_{12}$ 0 0 备用 0 1 PC 1 0 R1 1 1 X	2-4 译码器，互斥
输出分配	$\mu IR_{11}\mu IR_{10}\mu IR_9$ 0 0 0 备用 0 0 1 R0 0 1 0 R1 0 1 1 PC 1 0 0 IR 1 0 1 X	3-8 译码器，互斥
	1 1 0 R2 1 1 1 MAR	
存储器读写控制	$\mu IR_5\mu IR_4$ 0 0 备用 0 1 RD 1 0 WR 1 1 备用	2-4 译码器，互斥
后继微地址形成	$\mu IR_2\mu IR_1\mu IR_0$ 0 0 0 备用 0 0 1 $\mu PC+1$ ，顺序执行 0 1 0 JP 无条件转跳，地址由 $\mu IR_{31}\sim\mu IR_{24}$ 提供 0 1 1 QJP 高四位按操作码转移，低 4 位为 0 1 0 0 备用 1 0 1 备用 1 1 0 备用 1 1 1 备用	3-8 译码器，互斥
转跳地址	$\mu IR_{31} \mu IR_{30} \mu IR_{29} \mu IR_{28} \mu IR_{27} \mu IR_{26} \mu IR_{25} \mu IR_{24}$	

5.6 指令执行流程

(1) 编写程序

编写程序	地址	代码	地址	内容	结果
MOV1 5, R0	0	00001 000	1	0000 0101	
MOV2 3, R1	2	00010 000	3	0000 0011	
SUB R0, R1	4	00011 000			
MOV4 R1, (FF)	5	00100 000	6	1111 1111	2

计算机组成与设计课程设计报告

ADD R0, R1	7	00110 000			
MOV4 R1, (FE)	8	00100 000	9	1111 1110	7
MUL R0, R1	10	01000 000			
MOV4 R1, (FD)	11	00100 000	12	1111 1101	23
MUL R0, R1, (FC)	13	01001 000	14	1111 1100	AF
AND R0, R1	15	01010 000			
MOV4 R1, (FB)	16	00100 000	17	1111 1011	01
OR R0, R1	18	01011 000			
MOV4 R1, (FA)	19	00100 000	20	1111 1010	05
NOR R0, R1	21	01100 000			
MOV4 R1, (F9)	22	00100 000	23	1111 1001	FA
XOR R0, R1	24	01101 000			
MOV4 R1, (F8)	25	00100 000	26	1111 1000	FF
ADD_SELF R0	27	01110 000			
MOV3 R0, (F7)	28	01111 000	29	1111 0111	06
SUB_SELF R0	30	10000 000			
MOV3 R0, (F6)	31	01111 000	32	1111 0110	05
NOT_SELF R0	33	10001 000			
MOV3 R0, (F5)	34	01111 000	35	1111 0101	FA
LSL R0	36	10010 000			
MOV3 R0, (F4)	37	01111 000	38	1111 0100	F4
LSR R0	39	10011 000			
MOV3 R0, (F3)	40	01111 000	41	1111 0011	7A
MOV5 R0, ((C0))	42	10100 000	43	1100 0000	
MOV6 R1, ((C1))	44	10101 000	45	1100 0001	
MOV5 PC, (E0)	46	00101 000	47	1110 0000	
MOV6 RAM(70), PC	48	00111 000	49	0111 0000	
MOV5 PC, (E0)	50	00101 000	51	1110 0000	
MOV6 RAM(90), PC	52	00111 000	53	1001 0000	
MOV5 PC, (E0)	54	00101 000	55	1110 0000	
MOV6 RAM(50), PC	56	00111 000	57	0101 0000	
HALT	58	11111 000			
MOV1 4, R0	70	00001 000	71	0000 0100	
MOV2 0, R1	72	00010 000	73	0000 0000	
MOV7 0, X	74	10110 000	75	0000 0000	
MOV8 PC, R2	76	10111 000			
MOV9 R2, PC	77	11000 000			
ADD X, D0	78	11001 000	79	1101 0000	
ADD_SELF X	7A	11010 000			
JNZ	7B	11011 000			
MOV4 R1, (EE)	7C	00100 000	7D	1110 1110	19
ADD_SELF R0	7E	01110 000			

计算机组成与设计课程设计报告

MOV3 R0, (ED)	7F	01111 000	80	1110 1101	05
DIV R1, R0, (EC)	81	11100 000	82	1110 1100	05
MOV12 (E0), PC	83	11110 000	84	1110 0000	
MOV1 4, R0	90	00001 000	91	0000 0100	
MOV2 1, R1	92	00010 000	93	0000 0001	
MOV7 0, X	94	10110 000	95	0000 0000	
MOV8 PC, R2	96	10111 000			
MOV9 R2, PC	97	11000 000			
MUL X, B0	98	11101 000	99	1011 0000	
ADD_SELF X	9A				
JNZ	9B				
MOV4 R1, (C4)	9C	00100 000	9D	1100 0100	78
ADD_SELF R0	9E	01110 000			
MOV3 R0, (C5)	9F	01111 000	A0	1110 1101	05
DIV R1, R0, (C6)	A1	11100 000	A2	1110 1100	18
MOV12 (E0), PC	A3	11110 000	A4	1110 0000	
MOV1 2, R0	50	00001 000	51	0000 0010	
MOV2 1, R1	52	00010 000	53	0000 0001	
MOV7 0, X	54	10110 000	55	0000 0000	
MOV8 PC, R2	56	10111 000			
MOV9 R2, PC	57	11000 000			
MOV1 5, R0	58	00001 000	59	0000 0101	
MUL R0, R1	5A	01000 000			
MOV1 2, R0	5B	00001 000	5C	0000 0010	
ADD_SELF X	5D	11010 000			
JNZ	5E	11011 000			
MOV4 R1, (CA)	5F	00100 000	60	1100 1010	7D
ADD_SELF R0	61	01110 000			
MOV3 R0, (CB)	62	01111 000	63	1100 1011	03
DIV R1, R0, (CC)	64	11100 000	65	1100 1100	29
MOV12 (E0), PC	66	11110 000	67	1110 0000	

三个函数解释：

Function1（求平均值）：计算 $(D0+D1+D2+D3+D4)/5$ ，其中 D0、D1、D2、D3、D4 是 RAM 中的连续单元。

计算机组成与设计课程设计报告

MOV1 4,R0
MOV2 0,R1
MOV7 0,X
MOV8 PC,R2
MOV9 R2,PC
ADD X,D0
ADD_SELF X
JNZ
MOV4 R1,(EE)
ADD_SELF R0
MOV3 R0,(ED)
DIV R1,R0,(EC)

①初始化 X, R1, R0

X = 0; //变址寄存器, 通过自增, 来实现地址递增

R1 = 0; //作为 ACC 累加器

R0 = 4; //循环次数-1, 假设循环次数为 5

②保存 PC 到 R2, 满足循环条件时需要强制转跳, 就需要通过 R2 + 1 来更新 PC (因为 PC 里存的是下一条指令的地址, 所以需要+1)。

③进入循环

do

{

temp=(D0+(X)); //D0 是 16 进制

R1=R1+temp;

(X)++;

}while(R0-X>=0)

通过 JNZ 指令来确定是否满足循环条件, 如果循环条件不满足, 那么将顺序执行, 如果满足, 那么就会强制转跳到循环开始的地方。JNZ 是通过减法实现的, 如果 $R0-X \geq 0$ 那么 flag 就会为 1, 为 1 则会将 3-8 译码器的输出强制转换为无条件转跳, 如果 $R0-X < 0$, 那么 flag 为 0, 那么将会顺序执行。

④求平均数

Avg=R1/(R0+1)

由于 R0 是循环次数减一, 所以在计算平均数的时候需要进行自增。

本函数可用于计算累加和, 可通过设置循环次数以及初始地址, 依次来实现从指定地址开始的累加和。

Function2（累乘）：计算 $(B0 * B1 * B2 * B3 * B4)$ ，其中 B0、B1、B2、B3、B4 是 RAM 中的连续单元。

MOV1 4, R0
MOV2 1, R1
MOV7 0, X
MOV8 PC, R2
MOV9 R2, PC
MUL X, B0
ADD_SELF X
JNZ
MOV4 R1, (C4)
ADD_SELF R0
MOV3 R0, (C5)
DIV R1, R0, (C6)
MOV12 (E0), PC

①初始化 X, R1, R0

X = 0; //变址寄存器，通过自增，来实现地址递增

R1 = 1; //作为累乘器

R0 = 4; //循环次数-1，假设循环次数为 5

②保存 PC 到 R2，满足循环条件时需要强制转跳，就需要通过 R2+1 来更新 PC（因为 PC 里存的是下一条指令的地址，所以需要+1）。

③进入循环

do

{

temp=(B0+(X)); //B0 是 16 进制

R1=R1 * temp;

(X)++;

}while(R0-X>=0)

通过 JNZ 指令来确定是否满足循环条件，如果循环条件不满足，那么将顺序执行，如果满足，那么就会强制转跳到循环开始的地方。JNZ 是通过减法实现的，如果 $R0-X \geq 0$ 那么 flag 就会为 1，为 1 则会将 3-8 译码器的输出强制转换为无条件转跳，如果 $R0-X < 0$ ，那么 flag 为 0，那么将会顺序执行。

本函数可用于计算累乘和，可通过设置循环次数以及初始地址，依次来实现从指定地址开始的累乘和。

Function3（求幂）：

MOV1 2, R0
MOV2 1, R1
MOV7 0, X
MOV8 PC, R2
MOV9 R2, PC
MOV1 5, R0
MUL R0, R1
MOV1 2, R0
ADD_SELF X
JNZ
MOV4 R1, (CA)
ADD_SELF R0
MOV3 R0, (CB)
DIV R1, R0, (CC)
MOV12 (E0), PC

①初始化 X, R1, R0

X = 0; //通过自增, 来判断循环是否结束

R1 = 1; //作为累乘器

R0 = 2; //循环次数-1, 假设循环次数为 3 (本例中我们求 5 的 3 次方)

②保存 PC 到 R2, 满足循环条件时需要强制转跳, 就需要通过 R2+1 来更新 PC (因为 PC 里存的是下一条指令的地址, 所以需要+1)。

③进入循环

do

{

R0 = N; // N 是 x^y 中的 x

R1 = R1 * R0;

R0 = temp; // temp 是 x^y 中的 y

(X)++;

}while(R0-X>=0)

通过 JNZ 指令来确定是否满足循环条件, 如果循环条件不满足, 那么将顺序执行, 如果满足, 那么就会强制转跳到循环开始的地方。JNZ 是通过减法实现的, 如果 $R0-X \geq 0$ 那么 flag 就会为 1, 为 1 则会将 3-8 译码器的输出强制转换为无条件转跳, 如果 $R0-X < 0$, 那么 flag 为 0, 那么将会顺序执行。

本函数中可以实现对 x^y 的求解, 不过受限于 8 位 ALU 运算器, 无法求解 x 或者 y 较大时候的值。

将代码写入 RAM 中，如下：

addr	+0	+1	+2	+3	+4	+5	+6	+7
000	00001000	00000101	00010000	00000011	00011000	00100000	11111111	00110000
008	00100000	11111110	01000000	00100000	11111101	01001000	11111100	01010000
010	00100000	11111011	01011000	00100000	11111010	01100000	00100000	11111001
018	01101000	00100000	11111000	01110000	01111000	11110111	10000000	01111000
020	11110110	10001000	01111000	11110101	10010000	01111000	11110100	10011000
028	01111000	11110011	10100000	11000000	10101000	11000001	00101000	11100000
030	00111000	01110000	00101000	11100000	00111000	10010000	00101000	11100000
038	00111000	01010000	11111000	00000000	00000000	00000000	00000000	00000000
040	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
048	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
050	00001000	00000010	00010000	00000001	10110000	00000000	10111000	11000000
058	00001000	00000101	01000000	00001000	00000010	11010000	11011000	00100000
060	11001010	01110000	01111000	11001011	11100000	11001100	11110000	11100000
068	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
070	00001000	00000100	00010000	00000000	10110000	00000000	10111000	11000000
078	11001000	11010000	11010000	11011000	00100000	11101110	01110000	01111000
080	11101101	11100000	11101100	11110000	11100000	00000000	00000000	00000000
088	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
090	00001000	00000100	00010000	00000001	10110000	00000000	10111000	11000000
098	11101000	10110000	11010000	11011000	00100000	11000100	01110000	01111000
0a0	11000101	11100000	11000110	11110000	11100000	00000000	00000000	00000000
0a8	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0b0	00000001	00000010	00000011	00000100	00000101	00000000	00000000	00000000
0b8	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0c0	11110000	11101111	00000000	00000000	00000000	00000000	00000000	00000000
0c8	00000000	00000000	01111101	00000011	00000000	00000000	00000000	00000000
0d0	00000001	00000011	00000101	00000111	00001001	00000000	00000000	00000000
0d8	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0e0	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0e8	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0f0	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
0f8	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000

(2) 指令流程

首先读取指令，然后跳转到相应指令的入口地址，执行完成后，重新读取指令，循环下去，直到执行完停机指令。

指令及相应入口地址和微操作如下：

指令	入口地址	微指令
读指令	00000 000	RAM→IR
		PC+1→PC
		QJP
MOV1 N#, R0	00001 000	PC→MAR
		PC+1→PC
		RAM→R0
		PC→MAR
		JP
MOV2 N#, R1	00010 000	PC→MAR
		PC+1→PC
		RAM→R1
		PC→MAR
		JP
SUB R0, R1	00011 000	R0-R1→R1
		PC→MAR
		JP
MOV4 R1, (N#)	00100 000	PC→MAR
		PC+1→PC
		RAM→MAR
		R1→RAM
		PC→MAR
		JP
MOV5 PC, (N#)	00101 000	PC→MAR
		PC+1→PC
		RAM→MAR
		PC→RAM
		PC→MAR
		JP

计算机组成与设计课程设计报告

ADD R0, R1	00110 000	$R0+R1 \rightarrow R1$
		PC→MAR
		JP
MOV6 RAM, PC	00111 000	PC→MAR
		RAM→PC
		PC→MAR
		JP
MUL R0, R1	01000 000	$R0 \times R1 \rightarrow R1$
		PC→MAR
		JP
MUL R0, R1, (FA)	01001 000	PC→MAR
		PC+1→PC
		RAM→MAR
		$R0 \times R1 \rightarrow (FA)$
		PC→MAR
		JP
AND R0, R1	01010 000	$R0 \& R1 \rightarrow R1$
		PC→MAR
		JP
OR R0, R1	01011 000	$R0 R1 \rightarrow R1$
		PC→MAR
		JP
NOR R0, R1	01100 000	$\neg (R0+R1) \rightarrow R1$
		PC→MAR
		JP
XOR R0, R1	01101 000	$R0 \oplus R1 \rightarrow R1$
		PC→MAR
		JP
ADD_SELF R0	01101 000	$R0+1 \rightarrow R0$
		PC→MAR
		JP
MOV3 R0, (N#)	01111 000	PC→MAR

计算机组成与设计课程设计报告

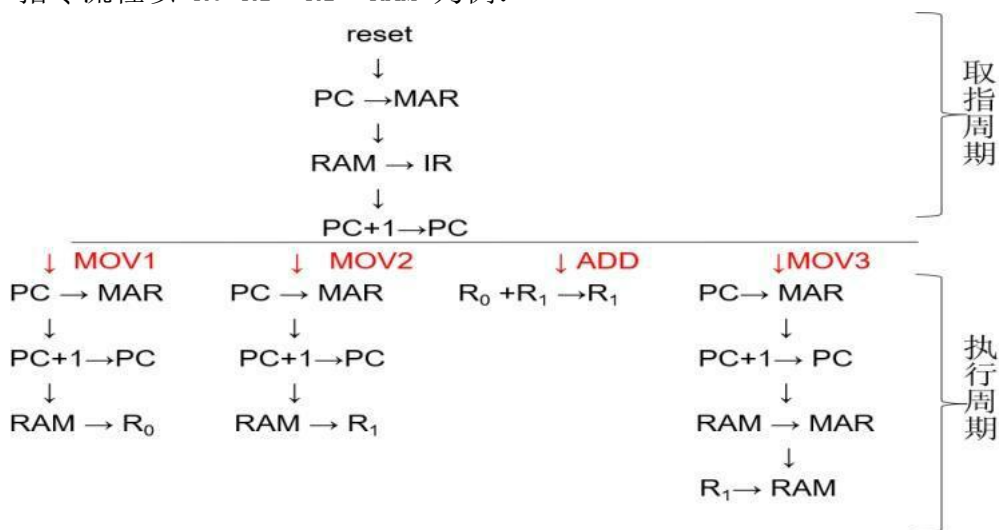
		PC+1→PC
		RAM→MAR
		RO→RAM
		PC→MAR
		JP
SUB_SELF RO	10000 000	RO-1→RO
		PC→MAR
		JP
NOT_SELF RO	10001 000	/RO→RO
		PC→MAR
		JP
LSL RO	10010 000	(1<-RO)→RO
		PC→MAR
		JP
LSR RO	10011 000	(RO→1)→RO
		PC→MAR
		JP
MOV7 RO, ((C0))	10100 000	PC→MAR
		PC+1→PC
		RAM→MAR
		RAM→MAR
		RO→RAM
		PC→MAR
		JP
MOV8 R1, ((C1))	10101 000	PC→MAR
		PC+1→PC
		RAM→MAR
		RAM→MAR
		R1→RAM
		PC→MAR
		JP
MOV9 N#, X	10110 000	PC→MAR
		PC+1→PC
		RAM→X
		PC→MAR

计算机组成与设计课程设计报告

		JP
MOV10 PC, R2	10111 000	PC→R2
		PC→MAR
		JP
MOV11 R2, PC	11000 000	R2+1→PC
		PC→MAR
		JP
ADD X, D0	11001 000	PC→MAR
		PC+1→PC
		RAM+X→MAR
		RAM+R1→R1
		PC→MAR
		JP
ADD_SELF X	11010 000	X+1→X
		PC→MAR
		JP
JNZ	11011 000	JNZ
		PC→MAR
		JP
DIV R1, R0, (N#)	11100 000	PC→MAR
		PC+1→PC
		RAM→MAR
		R1/R0→RAM
		PC→MAR
		JP
MUL X, B0	11101 000	PC→MAR
		PC+1→PC
		RAM+X→MAR
		RAM*R1→R1
		PC→MAR
		JP
MOV12 (N#), PC	11110 000	PC→MAR
		PC+1→PC
		RAM→MAR

		RAM→PC
		PC+1→PC
		PC+1→PC
		PC→MAR
		JP
HALT	11111 000	G=1

指令流程以 $R_0+R_1 \rightarrow R_1 \rightarrow \text{RAM}$ 为例：



5.7 编制微程序

- 根据指令流程和微指令格式就可以开始编制微程序。
- 指令流程中每一个流程对应一条微指令，写出这个流程所对应的数据通路的控制命令。例 $\text{RAM} \rightarrow \text{IR}$ 所需的控制命令是 $\text{RAM}, \text{RD}, \text{CPIR}$ ，并在下表中的相应位置填写上“1”，不需要的命令填写“0”。
- 另外每一条微指令都要确定下条微指令地址的生成方式。

微程序如下：

计算机组成与设计课程设计报告

程序序	操作	地址	R2R31~R2R30~R2R29~R2R28~R2R27~R2R26~R2R25~R2R24				R2R23	R2R22	R1R21~R1R20~R1R19~R1R18~R1R17~R1R16				A选择	B选择	输出分配	S0比较	R2R7	R1R6	R2R5~R4	R1R3~R2R1~R1R0	十六进制代码				
			具体地址				控制位操作S7	控制位操作S4	M	S3	S2	S1	S0	C0					左/右移S5(1为左)	RD	WR	常机	转移方式		
读指令	RAM→R2R	00000000					0	0				111110		10	00	100	0	0		10	0	001	3E	88	21
	PC+1→PC	00000001					0	0				010010		00	00	011	0	0		00	0	001	12	16	01
	J2P	00000010					0	0				000000		00	00	000	0	0		00	0	011	00	00	03
MOV1 R0, R0	PC→MAR	00001000					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	PC+1→PC	00001001					0	0				010010		00	01	011	0	0		00	0	001	12	16	01
	RAM→R0	00001010					0	0				111110		10	00	001	0	0		10	0	001	3E	8E	21
	PC→MAR	00001011					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	00001100					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
MOV2 R0, R1	PC→MAR	00010000					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	PC+1→PC	00010001					0	0				010010		00	01	011	0	0		00	0	001	12	16	01
	RAM→R1	00010010					0	0				111110		10	00	010	0	0		10	0	001	3E	84	21
	PC→MAR	00010011					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	00010100					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
SUB R0, R1	R0→R1→R1	00011000					0	0				001100		01	10	010	0	0		00	0	001	0C	64	01
	PC→MAR	00011001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	00011010					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
MOV4 R1, (R0)	PC→MAR	00100000					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	PC+1→PC	00100001					0	0				010010		00	01	011	0	0		00	0	001	12	16	01
	RAM→MAR	00100010					0	0				111110		10	00	111	0	0		10	0	001	3E	8E	21
	R1→RAM	00100011					0	0				110100		00	10	000	0	0		01	0	001	34	20	11
	PC→MAR	00100100					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
MOV5 PC, (R0)	JF	00100101					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
	PC→MAR	00101000					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	PC+1→PC	00101001					0	0				010010		00	01	011	0	0		00	0	001	12	16	01
	RAM→MAR	00101010					0	0				111110		10	00	111	0	0		10	0	001	3E	8E	21
	PC→RAM	00101011					0	0				110100		00	01	000	0	0		01	0	001	34	10	11
ADD R0, R1	PC→MAR	00101100					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	00101101					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
	R0+R1→R1	00110000					0	0				010000		01	10	010	0	0		00	0	001	13	64	01
	PC→MAR	00110001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	00110010					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
MOV6 RAM(R0), PC	PC→MAR	00111000					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	RAM→PC	00111001					0	0				111110		10	00	011	0	0		10	0	001	3E	8E	21
	PC→MAR	00111010					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	00111011					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
							0	0																	
MUL R0, R1	R0→R1→R1	01000000					0	1				000000		01	10	010	0	0		00	0	001	40	64	01
	PC→MAR	01000001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	01000010					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
							0	0																	
							0	0																	
MUL R0, R1 (R0)	PC→MAR	01001000					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	PC+1→PC	01001001					0	0				010010		00	01	011	0	0		00	0	001	12	16	01
	RAM→MAR	01001010					0	0				111110		10	00	111	0	0		10	0	001	3E	8E	21
	R0+R1→(FA)	01001011					0	1				000000		01	10	000	0	0		01	0	001	40	60	11
	PC→MAR	01001100					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
AND R0, R1	JF	01001101					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
	RO+R1→R1	01010000					0	0				110110		01	10	010	0	0		00	0	001	36	64	01
	PC→MAR	01010001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	01010010					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
							0	0																	
OR R0, R1	RO[R1→R1	01011000					0	0				111100		01	10	010	0	0		00	0	001	3C	64	01
	PC→MAR	01011001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	01011010					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
NOR R0, R1	/ (R0+R1)→R1	01100000					0	0				100010		01	10	010	0	0		00	0	001	22	64	01
	PC→MAR	01100001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	01100010					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
XOR R0, R1	RO+R1→R1	01101000					0	0				101100		01	10	010	0	0		00	0	001	2C	64	01
	PC→MAR	01101001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	01101010					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
ADD_SELF R0	R0+1→R0	01110000					0	0				000000		01	00	001	0	0		00	0	001	00	42	01
	PC→MAR	01110001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	01110010					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
MOV3 R0, (R0)	PC→MAR	01111000					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	PC+1→PC	01111001					0	0				010010		00	01	011	0	0		00	0	001	12	16	01
	RAM→MAR	01111010					0	0				111110		10	00	111	0	0		10	0	001	3E	8E	21
	RO→RAM	01111011					0	0				110110		01	00	000	0	0		01	0	001	34	40	11
	PC→MAR	01111100					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
SUB_SELF R0	JF	01111101					0	0				000000		00	00	000	0	0		00	0	010	00	00	02
							0	0																	
							0	0																	
							0	0																	
							0	0																	
NOT_SELF R0	/R0→R0	10001000					0	0				100000		01	00	001	0	0		00	0	001	20	42	01
	PC→MAR	10001001					0	0				110100		00	01	111	0	0		00	0	001	34	1E	01
	JF	10001010					0	0				000000		00	00	000	0	0		00	0	010	00	00	

计算机组成与设计课程设计报告

MOV11 R2, PC	R2+1->PC	11000000	00000000	0	0	000000	11	00	011	0	0	0	0	0	001	00	C6	01
	PC->R2	11000001	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	JP	11000010	00000000	0	0	000000	00	00	000	0	0	0	0	0	010	00	00	02
ADD X, R2	PC->R2	11001000	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	PC+1->PC	11001001	00000000	0	0	010010	00	01	011	0	0	0	0	0	001	12	16	01
	RAM+X->R2	11001010	00000000	0	0	010011	10	11	111	0	0	0	0	0	001	13	AE	21
	RAM+R1->R1	11001011	00000000	0	0	010011	10	10	010	0	0	0	0	0	001	13	AA	21
	PC->R2	11001100	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	JP	11001101	00000000	0	0	000000	00	00	000	0	0	0	0	0	010	00	00	02
ADD_SELF X	X+1->X	11010000	00000000	0	0	010010	00	11	101	0	0	0	0	0	001	12	3A	01
	PC->R2	11010001	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	JP	11010010	00000000	0	0	000000	00	00	000	0	0	0	0	0	010	00	00	02
JNZ	JNZ	11011000	11000000	0	0	001100	01	11	000	1	0	0	0	0	001	C00C	71	01
	PC->R2	11011001	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	JP	11011010	00000000	0	0	000000	00	00	000	0	0	0	0	0	010	00	00	02
DIV R1, R0 (NZ)	PC->R2	11100000	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	PC+1->PC	11100001	00000000	0	0	010010	00	01	011	0	0	0	0	0	001	12	16	01
	RAM->R2	11100010	00000000	0	0	111110	10	00	111	0	0	0	0	0	001	3E	8E	21
	R1/R0->RAM	11100011	00000000	0	0	000000	01	10	000	0	1	0	0	0	001	00	60	91
	PC->R2	11100100	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	JP	11100101	00000000	0	0	000000	00	00	000	0	0	0	0	0	010	00	00	02
MUL X, R2	PC->R2	11101000	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	PC+1->PC	11101001	00000000	0	0	010010	00	01	011	0	0	0	0	0	001	12	16	01
	RAM+X->R2	11101010	00000000	0	0	010011	10	11	111	0	0	0	0	0	001	13	AE	21
	RAM+R1->R1	11101011	00000000	0	1	000000	10	10	010	0	0	0	0	0	001	40	84	21
	PC->R2	11101100	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	JP	11101101	00000000	0	0	000000	00	00	000	0	0	0	0	0	010	00	00	02
MOV12 (NZ), PC	PC->R2	11110000	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	PC+1->PC	11110001	00000000	0	0	010010	00	01	011	0	0	0	0	0	001	12	16	01
	RAM->R2	11110010	00000000	0	0	111110	10	00	111	0	0	0	0	0	001	3E	8E	21
	RAM->PC	11110101	00000000	0	0	111110	10	00	011	0	0	0	0	0	001	3E	8E	21
	PC+1->PC	11110100	00000000	0	0	010010	00	01	011	0	0	0	0	0	001	12	16	01
	PC+1->PC	11110101	00000000	0	0	010010	00	01	011	0	0	0	0	0	001	12	16	01
	PC->R2	11111100	00000000	0	0	110100	00	01	111	0	0	0	0	0	001	34	1E	01
	JP	11111101	00000000	0	0	000000	00	00	000	0	0	0	0	0	010	00	00	02
HALT	G=1	11111000	00000000	0	0	000000	00	00	000	0	0	0	0	0	000	00	00	08

将微代码写入 ROM 中，如下图：

Addr	+0	+1	+2	+3	+4	+5	+6	+7
000	003E8821	00121601	00000003	00000000	00000000	00000000	00000000	00000000
008	00341E01	00121601	003E8221	00341E01	00000002	00000000	00000000	00000000
010	00341E01	00121601	003E8421	00341E01	00000002	00000000	00000000	00000000
018	000C6401	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
020	00341E01	00121601	003E8E21	00342011	00341E01	00000002	00000000	00000000
028	00341E01	00121601	003E8E21	00341011	00341E01	00000002	00000000	00000000
030	00136401	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
038	00341E01	003E8621	00341E01	00000002	00000000	00000000	00000000	00000000
040	00406401	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
048	00341E01	00121601	003E8E21	00406011	00341E01	00000002	00000000	00000000
050	00366401	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
058	003C6401	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
060	00226401	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
068	002C6401	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
070	00004201	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
078	00341E01	00121601	003E8E21	003E4011	00341E01	00000002	00000000	00000000
080	001F4201	00341E01	00000002	00000000	00000000	00000000	00000000	00000000

00204201	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
00804241	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
00804201	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
00341E01	00121601	003E8E21	003E8E21	003E4011	00341E01	00000002	00000000
00341E01	00121601	003E8E21	003E8E21	00342011	00341E01	00000002	00000000
00341E01	00121601	003E8A21	00341E01	00000002	00000000	00000000	00000000
00341C01	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
0000C601	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
00341E01	00121601	0013BE21	0013A421	00341E01	00000002	00000000	00000000
00123A01	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
C00C7...	00341E01	00000002	00000000	00000000	00000000	00000000	00000000
00341E01	00121601	003E8E21	00006091	00341E01	00000002	00000000	00000000
00341E01	00121601	0013BE21	0040A421	00341E01	00000002	00000000	00000000
00341E01	00121601	003E8E21	003E8621	00121601	00121601	00341...	00000002
00000008	00000000	00000000	00000000	00000000	00000000	00000000	00000000

5.8 完整电路结构



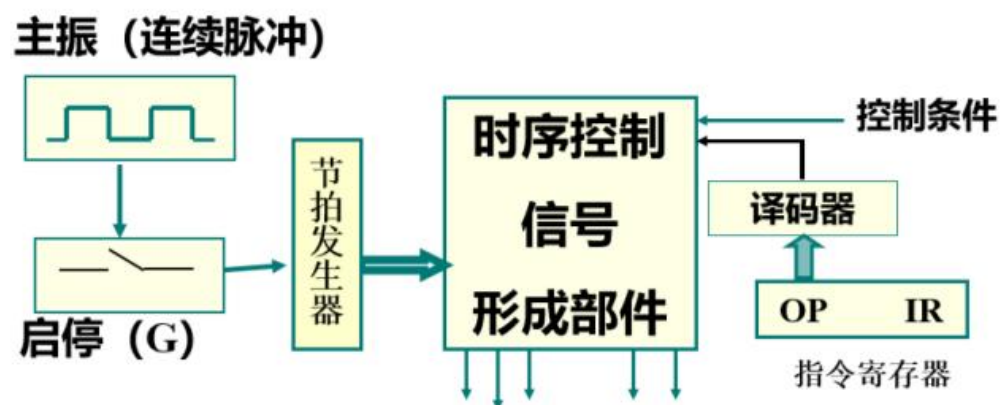
六、硬布线实现的模型机内核

6.1 确定模型机的功能

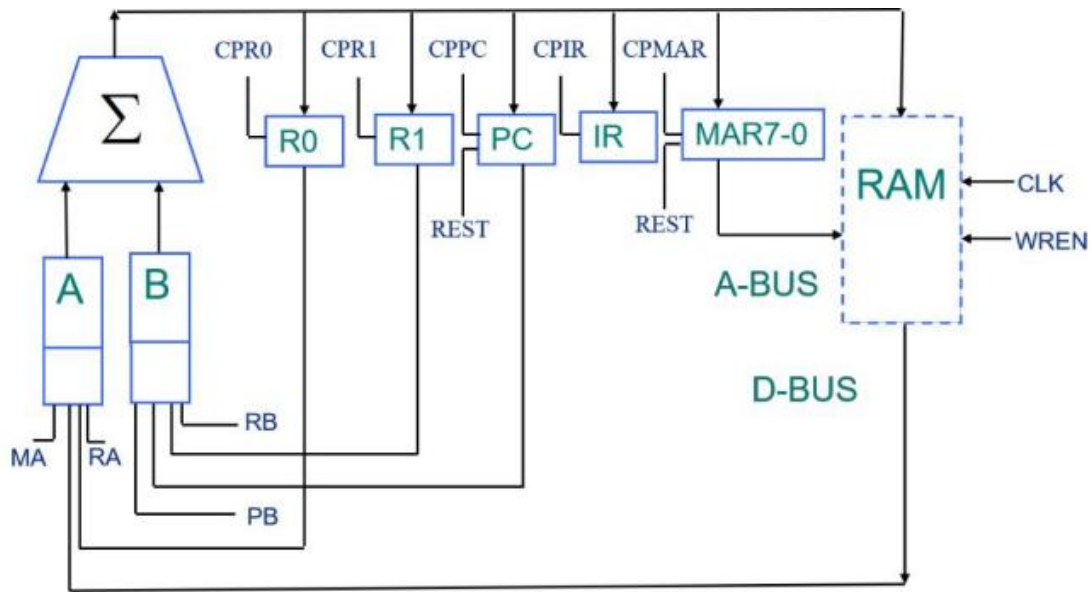
- (1) 硬布线模型机应实现以下功能：加、减、乘、除、与、或、非、异或、或非、自增、自减、逻辑左移、逻辑右移。
- (2) 同时硬布线模型机应支持多种寻址方式：立即数寻址、寄存器寻址等。

6.2 硬布线实现的 CU 框图及详解

6.2.1 CU 结构图



硬布线结构框图如下：



(2) 寄存器组的设置:

R_0 、 R_1 为通用寄存器:8 位;

IR 为指令寄存器:8 位;

PC 程序计数器:8 位;

MAR 为地址寄存器:8 位。

(3) 加法器 ALU 的设置

采用 74181、74182 实现以下功能;

方式	M=1 逻辑运算	M=0 算术运算	
S3 S2 S1 S0	逻辑运算	CN=1 (无进位)	CN=0 (有进位)
0 0 0 0	$F=A$	$F=A$	$F=A$ 加 1
0 0 0 1	$F=A+B$	$F=A+B$	$F=(A+B)$ 加 1
0 0 1 0	$F=A/B$	$F=A/B$	$F=(A/B)$ 加 1
0 0 1 1	$F=0$	$F=0$	$F=0$
0 1 0 0	$F=A \oplus B$	$F=A \oplus B$	$F=A \oplus B$ 加 1
0 1 0 1	$F=A/B$	$F=(A+B) \oplus A/B$	$F=(A+B) \oplus A/B$ 加 1
0 1 1 0	$F=A \oplus B$	$F=A \oplus B$ 减 1	$F=A \oplus B$
0 1 1 1	$F=A/B$	$F=A/B$ 减 1	$F=A/B$
1 0 0 0	$F=A+B$	$F=A+B$	$F=A+B$ 加 1
1 0 0 1	$F=A \oplus B$	$F=A+B$	$F=A+B$ 加 1
1 0 1 0	$F=B$	$F=(A+B) \oplus AB$	$F=(A+B) \oplus AB$ 加 1
1 0 1 1	$F=AB$	$F=AB$ 减 1	$F=AB$
1 1 0 0	$F=1$	$F=A \oplus A$	$F=A \oplus A$ 加 1
1 1 0 1	$F=A/B$	$F=(A+B) \oplus A$	$F=(A+B) \oplus A$ 加 1
1 1 1 0	$F=A+B$	$F=(A/B) \oplus A$	$F=(A/B) \oplus A$ 加 1
1 1 1 1	$F=A$	$F=A$ 减 1	$F=A$

(上表中的 “/” 表示求反)

采用 74283、74284、74285，实现乘法功能；

采用基本与或门，实现移位功能；

采用系统调用，实现除法功能；

(4) 选择器的设置

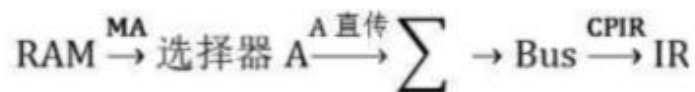
连入 A 选择器的数据来源是 RAM 的读出数据和 R_0 寄存器的数据。

连入 B 选择器的数据来源是 PC 的数据和 R_1 的数据。

(5) 数据通路

模型机的数据通路是以总线为基础, 以 CPU 为核心构成的。

① 取指令



② 送指令地址

$$\overset{PC}{PC} \rightarrow \text{选择器 B} \xrightarrow{B \text{ 直传}} \sum \rightarrow \text{Bus} \xrightarrow{CPMAR} \text{MAR}$$

③ 指令计数器+1

$$\overset{PC}{PC} \rightarrow \text{选择器 B} \xrightarrow{A+B+1} \sum \rightarrow \text{Bus} \xrightarrow{CPPC} PC$$

④ RAM→R(R 指任意一个寄存器)

$$\overset{MA}{RAM} \rightarrow \text{选择器 A/B} \xrightarrow{A/B \text{ 直传}} \sum \rightarrow \text{Bus} \xrightarrow{CPR} R$$

⑤ R→RAM(R 指任意一个寄存器)

$$\overset{R}{R} \rightarrow \text{选择器 A/B} \xrightarrow{A/B \text{ 直传}} \sum \rightarrow \text{Bus} \xrightarrow{WR} RAM$$

⑥ RA*RB→RC (RA 指选择器 A 控制的任意寄存器, RB 指选择器 B 控制的任意寄存器, C 指任意寄存器, *表示某种运算)

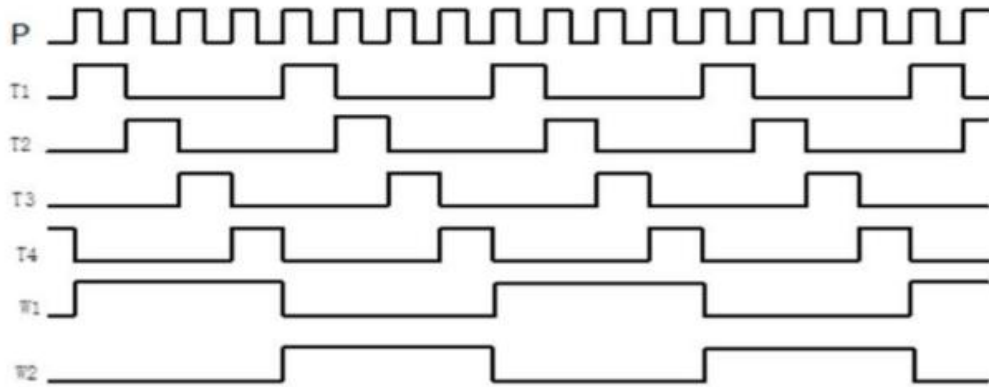
$$RA, RB \xrightarrow{RA, RB} \text{选择器 A、B} \xrightarrow{\text{运算}} \sum \rightarrow \text{Bus} \xrightarrow{RC} RC$$

⑦ RA*RB→RAM (RA 指选择器 A 控制的任意寄存器, RB 指选择器 B 控制的任意寄存器, *表示某种运算)

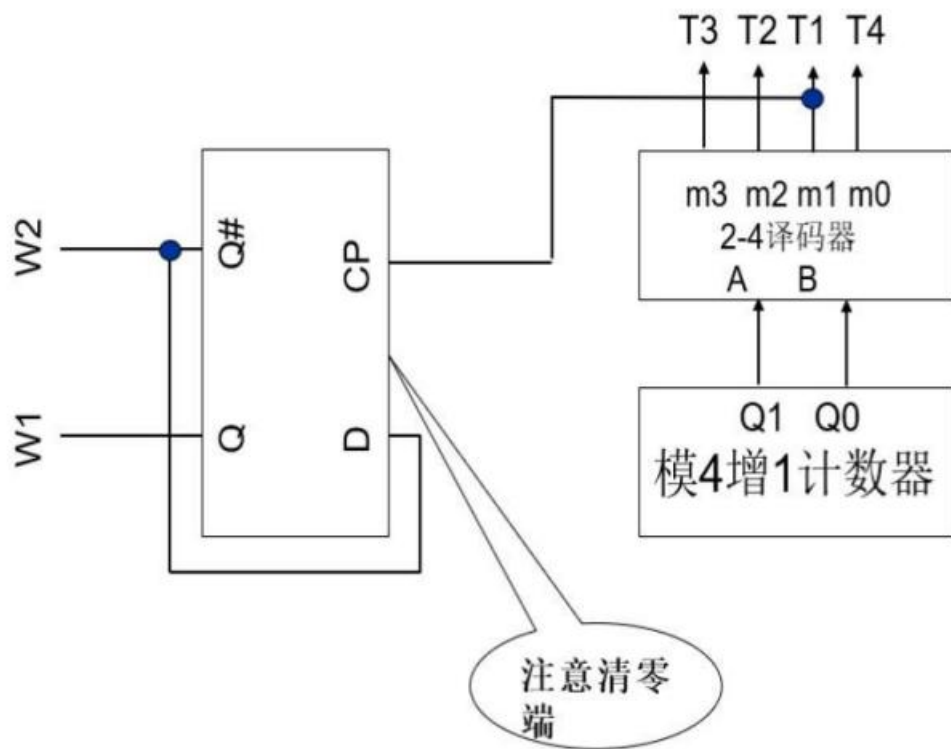
$$RA, RB \xrightarrow{RA, RB} \text{选择器 A、B} \xrightarrow{\text{运算}} \sum \rightarrow \text{Bus} \xrightarrow{WR} RAM$$

6.4 硬布线实现节拍发生器设计

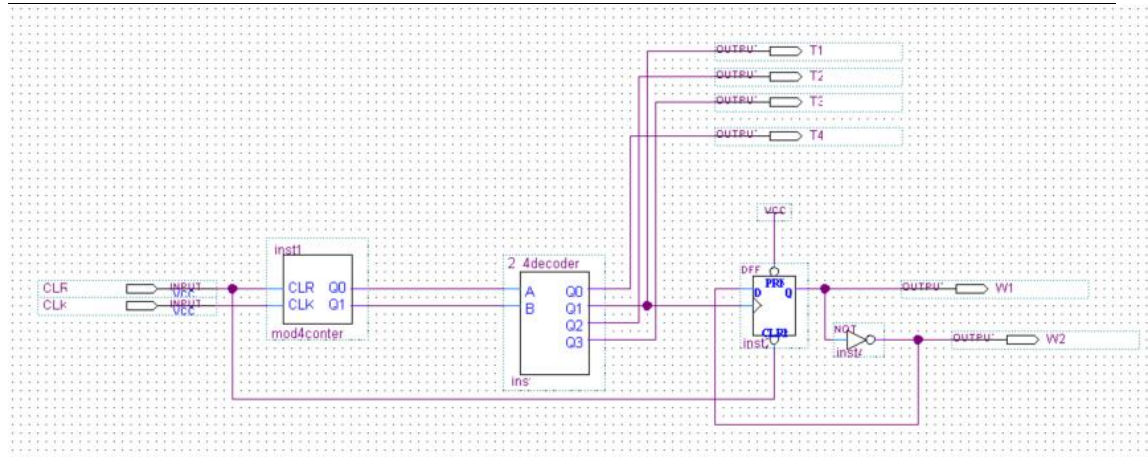
根据指令执行流程,设计时序为:分取指周期和执行周期,每个周期为 4 节拍,波形图如下:



W1 高电平时是取指周期的 4 节拍，W2 高电平时是执行周期的 4 节拍。节拍发生器实现电路框图如下：



节拍发生器电路图如下：



6.5 操作码 OP 译码器的设计

利用指令的操作码，得出逻辑表达式，使用逻辑表达式，利用与门、非门设计电路。

操作码：0000	$MOV1 = \overline{IR_7} \overline{IR_6} \overline{IR_5} \overline{IR_4}$
操作码：0001	$MOV2 = \overline{IR_7} \overline{IR_6} \overline{IR_5} IR_4$
操作码：0010	$ADD = \overline{IR_7} \overline{IR_6} IR_5 \overline{IR_4}$
操作码：0011	$MOV3 = \overline{IR_7} \overline{IR_6} IR_5 IR_4$
操作码：0100	$SUB = \overline{IR_7} IR_6 \overline{IR_5} \overline{IR_4}$
操作码：0101	$MUL = \overline{IR_7} IR_6 \overline{IR_5} IR_4$
操作码：0110	$AND = \overline{IR_7} IR_6 IR_5 \overline{IR_4}$
操作码：0111	$OR = \overline{IR_7} IR_6 IR_5 IR_4$
操作码：1000	$NOR = IR_7 \overline{IR_6} \overline{IR_5} \overline{IR_4}$
操作码：1001	$XOR = IR_7 \overline{IR_6} \overline{IR_5} IR_4$
操作码：1010	$ADD_SELF = IR_7 \overline{IR_6} IR_5 \overline{IR_4}$
操作码：1011	$SUB_SELF = IR_7 \overline{IR_6} IR_5 IR_4$
操作码：1100	$LSL = IR_7 IR_6 \overline{IR_5} \overline{IR_4}$

操作码：1101 $LSR = IR_7 IR_6 \overline{IR_5} IR_4$

操作码：1110 $DIV = IR_7 IR_6 IR_5 \overline{IR_4}$

操作码：1111 $HALT = IR_7 IR_6 IR_5 IR_4$

6.6 指令控制信号时序及逻辑设计

根据指令执行流程和数据通路，各控制信号的列表如下：

状态 指令	W1 (取值周期)					
	所有指令	MOV1	MOV2	ADD	MOV3	SUB
MA (选择RAM输出)	T2	T3	T3		T3	
RA (选择R0)				T1	T4	T1
PB (选择PC)	T1、T3	T1、T2	T1、T2		T1、T2	
RB (选择R1)				T1		T1
CPR0 (R0脉冲)				T1		T1
CPR1 (R1脉冲)			T3			
CRPC (PC脉冲)	T3	T2	T2		T2	
CPIR	T2					
CPMAR (MAR脉冲)	T1	T1	T1		T1、T3	
RD (读信号)	T2	T3	T3		T3	
WD (写信号)					T4	
G (停机)						
M	1(T1)、1(T2)、0(T3)	1(T1)、0(T2)、1(T3)	1(T1)、0(T2)、1(T3)	0(T1)	1(T1)、0(T2)、1(T3)、1(T4)	0(T1)
S3	1(T1)、1(T2)、1(T3)	1(T1)、1(T2)、1(T3)	1(T1)、1(T2)、1(T3)	1(T1)	1(T1)、1(T2)、1(T3)、1(T4)	0(T1)
S2	0(T1)、1(T2)、0(T3)	0(T1)、0(T2)、1(T3)	0(T1)、0(T2)、1(T3)	0(T1)	0(T1)、0(T2)、1(T3)、1(T4)	1(T1)
S1	1(T1)、1(T2)、0(T3)	1(T1)、0(T2)、1(T3)	1(T1)、0(T2)、1(T3)	0(T1)	1(T1)、0(T2)、1(T3)、1(T4)	1(T1)
S0	0(T1)、1(T2)、1(T3)	0(T1)、1(T2)、1(T3)	0(T1)、1(T2)、1(T3)	1(T1)	0(T1)、1(T2)、1(T3)、1(T4)	0(T1)
CN	0(T3)	0(T2)	0(T2)	1(T1)	0(T2)	0(T1)
S8 (控制除法)						
S7 (控制移位)						
S6 (选择移位寄存器OR1)						
S5 (选择左移右移1左)						
S4 (控制乘法)						

状态 指令	W1 (取值周期)						W2 (执行周期)			
	所有指令	MUL	AND	OR	NOR	XOR	ADD_SELF	SUB_SELF	LSL	LSR
MA (选择RAM输出)	T2									
RA (选择R0)		T1	T1	T1	T1	T1	T1	T1	T1	T1
PB (选择PC)	T1、T3									
RB (选择R1)		T1	T1	T1	T1	T1				
CPR0 (R0脉冲)		T1	T1	T1	T1	T1	T1	T1	T1	T1
CPR1 (R1脉冲)										
CRPC (PC脉冲)	T3									
CPIR	T2									
CPMAR (MAR脉冲)	T1									
RD (读信号)	T2									
WD (写信号)										
G (停机)										
M	1(T1)、1(T2)、0(T3)		1(T1)	1(T1)	1(T1)	1(T1)	0(T1)	0(T1)		
S3	1(T1)、1(T2)、1(T3)		1(T1)	1(T1)	0(T1)	0(T1)	0(T1)	1(T1)		
S2	0(T1)、1(T2)、0(T3)		0(T1)	1(T1)	0(T1)	1(T1)	0(T1)	1(T1)		
S1	1(T1)、1(T2)、0(T3)		1(T1)	1(T1)	0(T1)	1(T1)	0(T1)	1(T1)		
S0	0(T1)、1(T2)、1(T3)		1(T1)	0(T1)	1(T1)	0(T1)	0(T1)	1(T1)		
CN	0(T3)						0(T1)	1(T1)		
S8 (控制除法)									1(T1)	1(T1)
S7 (控制移位)									1(T1)	1(T1)
S6 (选择移位寄存器OR1)									1(T1)	1(T1)
S5 (选择左移右移1左)									1(T1)	0(T1)
S4 (控制乘法)		T1								

计算机组成与设计课程设计报告

状态 指令	W1 (取值周期)								W2 (执行周期)
MA (选择RAM输出)	所有指令	DIV	HALT						
RA (选择R0)	T2	T1							
PB (选择PC)	T1、T3								
RB (选择R1)		T1							
CPR0 (R0脉冲)		T1							
CPR1 (R1脉冲)									
CRPC (PC脉冲)	T3								
CPIR	T2								
CPMAR (MAR脉冲)	T1								
RD (读信号)	T2								
WD (写信号)									
G (停机)			T1						
M	1(T1) + 1(T2) + 0(T3)								
S3	1(T1) + 1(T2) + 1(T3)								
S2	0(T1) + 1(T2) + 0(T3)								
S1	1(T1) + 1(T2) + 0(T3)								
S0	0(T1) + 1(T2) + 1(T3)								
CH	0(T3)								
S8 (控制除法)		T1							
S7 (控制移位)									
S6 (选择移位寄存器OR1)									
S5 (选择左移右移1左)									
S4 (控制乘法)									

表中单元格内，填写的是该控制信号发生的条件

根据列表，各控制信号的逻辑表达式为：

$$MA = W1T2 + W1T3 \text{ (MOV1 + MOV2 + MOV3)}$$

$$RA = W2[T1(ADD + SUB + MUL + DIV + AND + OR + NOR + XOR + ADD_SELF + SUB_SELF + LSL + LSR) + T4MOV3]$$

$$PB = W1(T1 + T3) + W2(T1 + T2)(MOV1 + MOV2 + MOV3)$$

$$RB = W2T1(ADD + SUB + MUL + DIV + AND + OR + NOR + XOR)$$

$$CPR0 = W2[T3MOV1 + T1(ADD + SUB + MUL + DIV + AND + OR + NOR + XOR + ADD_SELF + SUB_SELF + LSL + LSR)]$$

$$CPR1 = W2T3MOV2$$

$$CPPC = W1T3 + W2T2(MOV1 + MOV2 + MOV3)$$

$$CPIR = W1T2$$

$$CPMAR = W1T1 + W2[T1(MOV1 + MOV2 + MOV3) + T3MOV3]$$

$$RD = W1T2 + W2T3(MOV1 + MOV2 + MOV3)$$

$$WR = W2T4MOV3$$

$$G = W2T1HALT$$

$$M = W1\overline{T3} + W2\overline{T2}(MOV1 + MOV2 + MOV3) + W2\overline{T1}(ADD + SUB + ADD_SELF + SUB_SELF) + W2T1(AND + OR + NOR + XOR)$$

$$S3 = W1 + W2(MOV1 + MOV2 + MOV3 + ADD + AND + OR + SUB_SELF) + W2\overline{T1}(SUB +$$

NOR + XOR + ADD_SELF)

$$S2 = W1\overline{T1}\overline{T3} + W2\overline{T1}\overline{T2}(MOV1 + MOV2 + MOV3) + W2\overline{T1}(ADD + AND + NOR + ADD_SELF) + W2T1(SUB + OR + XOR + SUB_SELF)$$

$$S1 = W1\overline{T3} + W2\overline{T2}(MOV1 + MOV2 + MOV3) + W2\overline{T1}(ADD + NOR + ADD_SELF) + W2T1(SUB + AND + OR + XOR + SUB_SELF)$$

$$S0 = W1\overline{T1} + W2\overline{T1}(MOV1 + MOV2 + MOV3 + SUB + OR + XOR + ADD_SELF) + W2T1(ADD + AND + NOR + SUB_SELF)$$

$$CN = W1\overline{T3} + W2\overline{T2}(MOV1 + MOV2 + MOV3) + W2T1(ADD + SUB_SELF) + W2\overline{T2}(SUB + ADD_SELF)$$

$$S8 = W2T1DIV$$

$$S7 = W2T1(LSL + LSR)$$

$$S6 = W2T1(LSL + LSR)$$

$$S5 = W2T1LSL + W2\overline{T1}LSR$$

$$S4 = W2T1MUL$$

其中每个信号的含义如下：

信号	解释
MA	选择 RAM 输出作为加法器的输入
RA	选择 R0 作为加法器的输入
PB	选择 PC 作为加法器的输入
RB	选择 R1 作为加法器的输入
CPR0	R0 的脉冲
CPR1	R1 的脉冲
CPPC	PC 的脉冲
CPIR	IR 的脉冲
CPMAR	MAR 的脉冲
RD	读信号
WR	写信号
G	停机
M	控制 ALU 运算
S3	控制 ALU 运算
S2	控制 ALU 运算
S1	控制 ALU 运算

S0	控制 ALU 运算
CN	控制 ALU 运算
S8 (控制除法)	1 表示进行除法
S7 (控制移位)	1 表示进行移位
S6 (选择移位寄存器 OR1)	选择移位寄存器 0 表示对 R1 进行移位
S5 (选择左移右移 1 左移)	1 表示左移 0 表示右移
S4 (控制乘法)	1 表示进行乘法

根据逻辑表达式，利用基本的与门、或门、非门实现控制信号即可。

该控制信号的实现，结合内核的数据通路，设计完整的电路图。

6.7 指令执行流程

(1) 指令种类

指令种类	解释
MOV1 N#, R0	N 送入 R0
MOV2 N#, R1	N 送入 R1
MOV3 R0, (N#)	R0 送 RAM
ADD R0, R1	ROR1 相加送 R0
SUB R0, R1	ROR1 相减送 R0
MUL R0, R1	ROR1 相乘送 R0
DIV R0, R1	ROR1 相除送 R0
AND R0, R1	R0 与 R1 送 R0
OR R0, R1	R0 或 R1 送 R0
NOR R0, R1	R0 或非 R1 相 R0
XOR R0, R1	R0 异或 R1 相 R0
ADD_SELF R0	R0+1 送 R0
SUB_SELF R0	R0-1 送 R0
LSL R0	R0 左移 1 位送 R0
LSR R0	R0 右移 1 位送 R0
HALT	停机

(2) 编写程序, 将代码写入 RAM

编写程序	地址	内容	地址	内容	结果
MOV1 5, R0	0	0000 0000	1	0000 0101	
MOV2 2, R1	2	0001 0000	3	0000 0010	
ADD R0, R1	4	0010 0000			7

计算机组成与设计课程设计报告

MOV3 R0, (FF)	5	0011 0000	6	1111 1111	
SUB R0, R1	7	0100 0000			5
MOV3 R0, (FE)	8	0011 0000	9	1111 1110	
MUL R0, R1	10	0101 0000			A
MOV3 R0, (FD)	11	0011 0000	12	1111 1101	
AND R0, R1	13	0110 0000			2
MOV3 R0, (FC)	14	0011 0000	15	1111 1100	
OR R0, R1	16	0111 0000			2
MOV3 R0, (FB)	17	0011 0000	18	1111 1011	
NOR R0, R1	19	1000 0000			FD
MOV3 R0, (FA)	20	0011 0000	21	1111 1010	
XOR R0, R1	22	1001 0000			FF
MOV3 R0, (F9)	23	0011 0000	24	1111 1001	
ADD_SELF R0	25	1010 0000			0
MOV3 R0, (F8)	26	0011 0000	27	1111 1000	
SUB_SELF R0	28	1011 0000			FF
MOV3 R0, (F7)	29	0011 0000	30	1111 0111	
LSL R0	31	1100 0000			FE
MOV3 R0, (F6)	32	0011 0000	33	1111 0110	
LSR R0	34	1101 0000			7F
MOV3 R0, (F5)	35	0011 0000	36	1111 0101	
DIV R0, R1	37	1110 0000			3F
MOV3 R0, (F4)	38	0011 0000	39	1111 0100	
HALT	40	1111 0000			

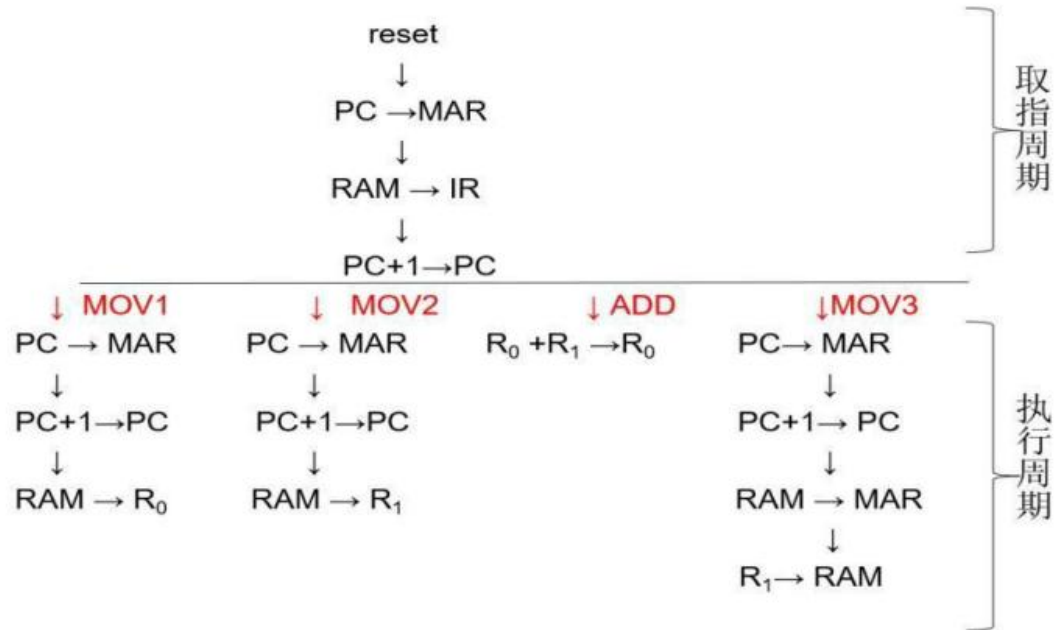
将代码写入 RAM 中，如下图：

Addr	+0	+1	+2	+3	+4	+5	+6	+7	ASCII
0	00000000	00000101	00010000	00000010	00100000	00110000	11111111	01000000	... 0. @
8	00110000	11111110	01010000	00110000	11111101	01100000	00110000	11111100	0.P0.0.
16	01110000	00110000	11111011	10000000	00110000	11111010	10010000	00110000	p0..0..0
24	11111001	10100000	00110000	11111000	10110000	00110000	11110111	11000000	..0..0..
32	00110000	11110110	11010000	00110000	11110101	11100000	00110000	11110100	0..0..0.
40	11110000	00000000	00000000	00000000	00000000	00000000	00000000	00000000	0..0..0..

首先读取指令，然后跳转到相应指令的入口地址，执行完成后，重新读取指令，

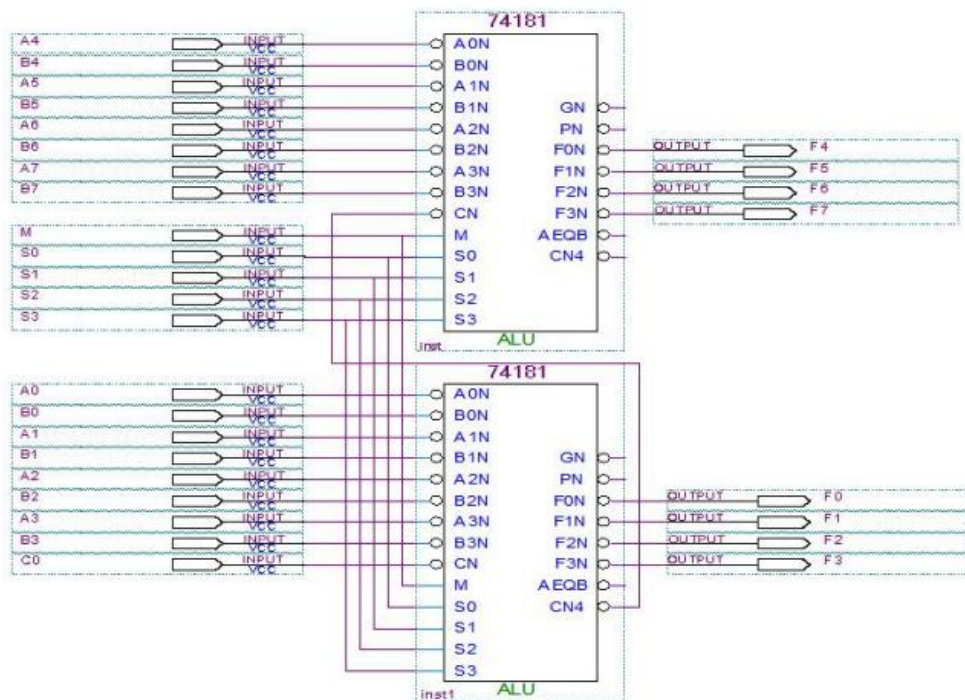
循环下去,直到执行完停机指令。

指令流程以 $R_0+R_1 \rightarrow R_0 \rightarrow \text{RAM}$ 为例

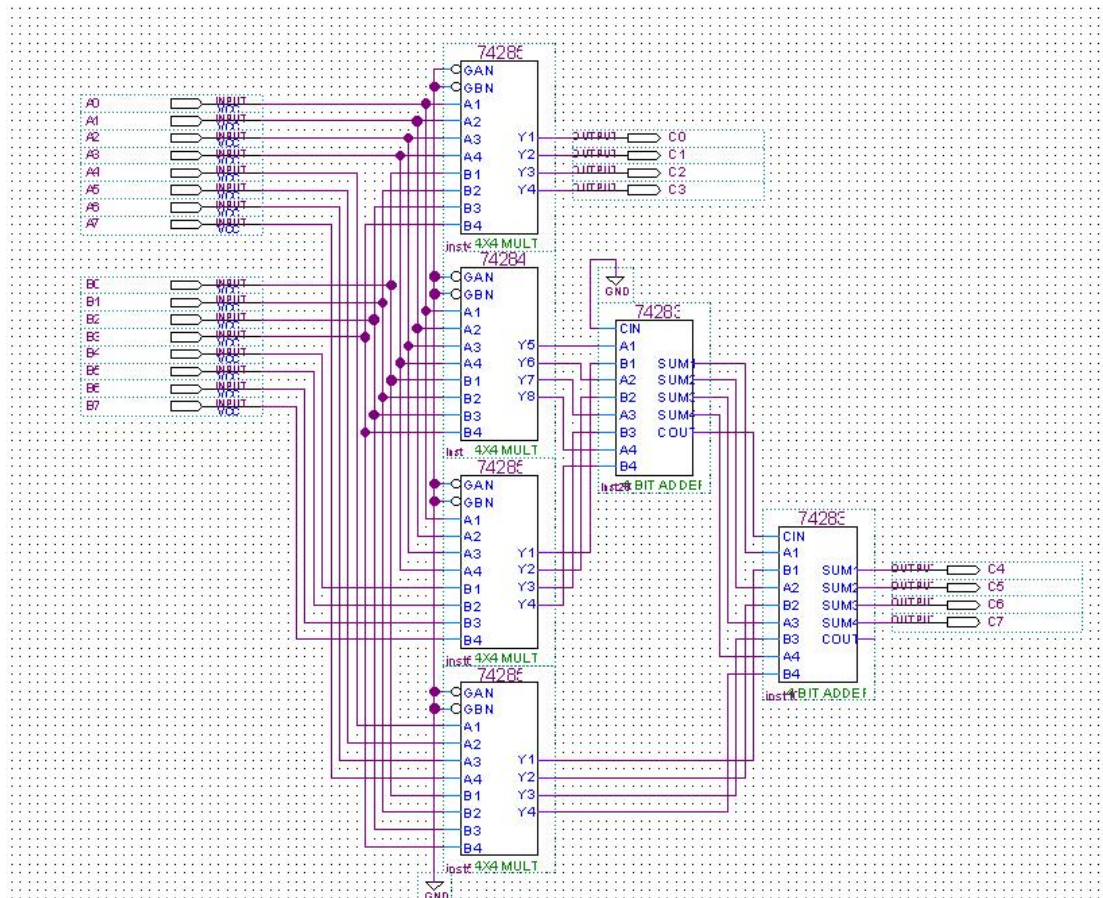


6.8 分部件结构

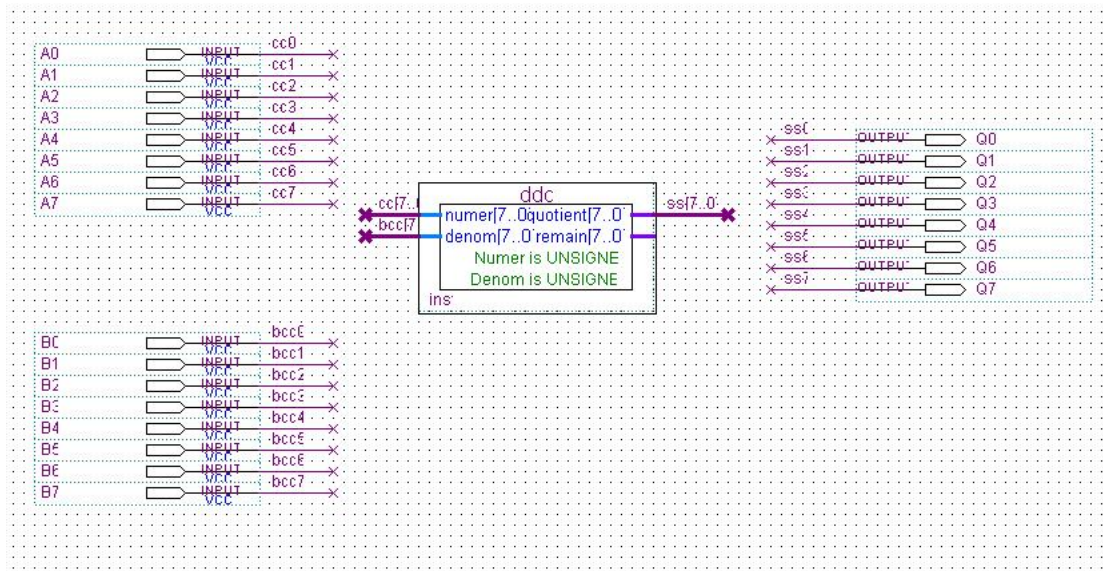
(1) 基本加减逻辑运算器件



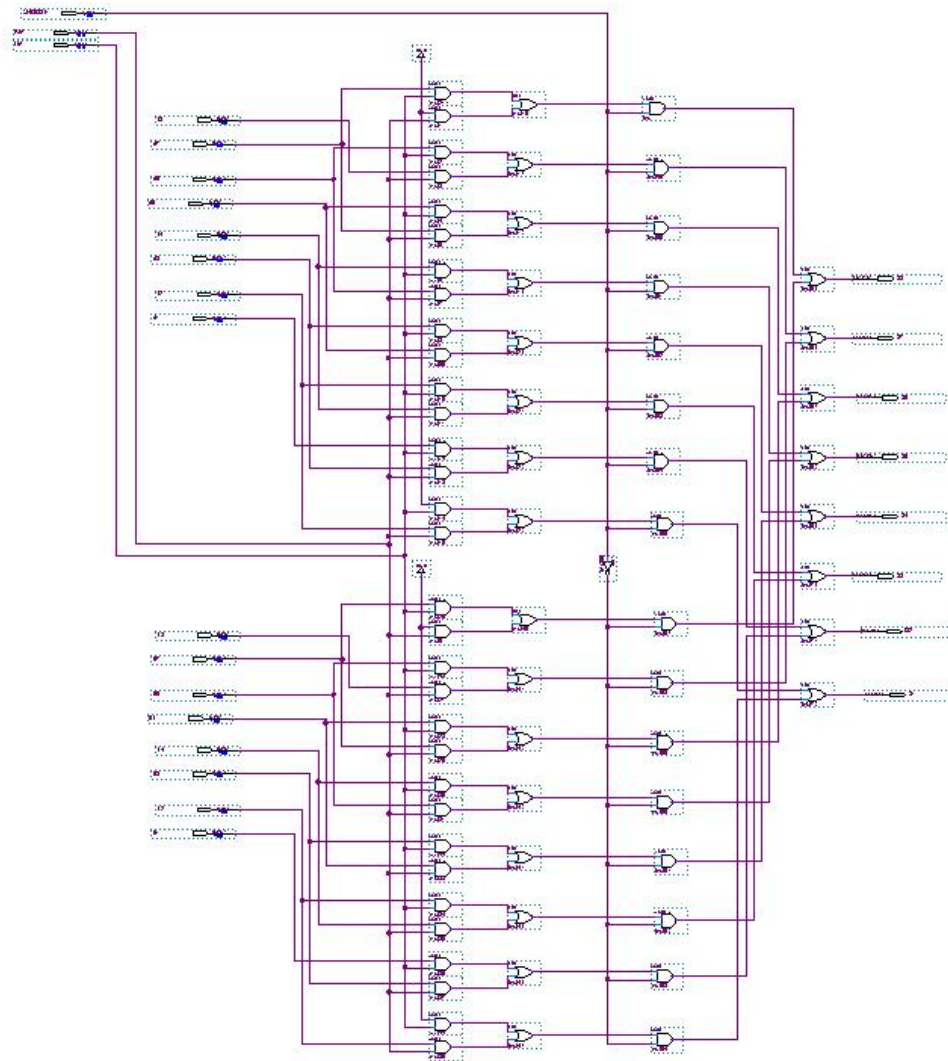
(2) 乘法器



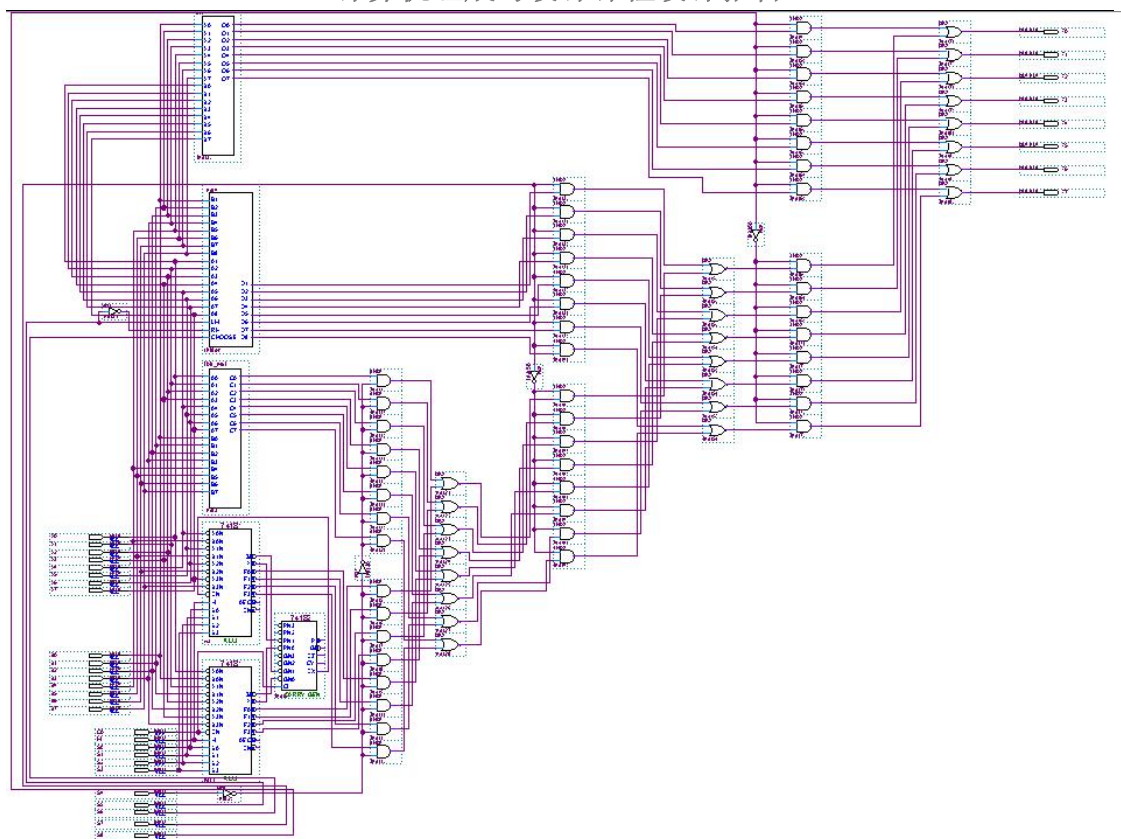
(3) 除法器



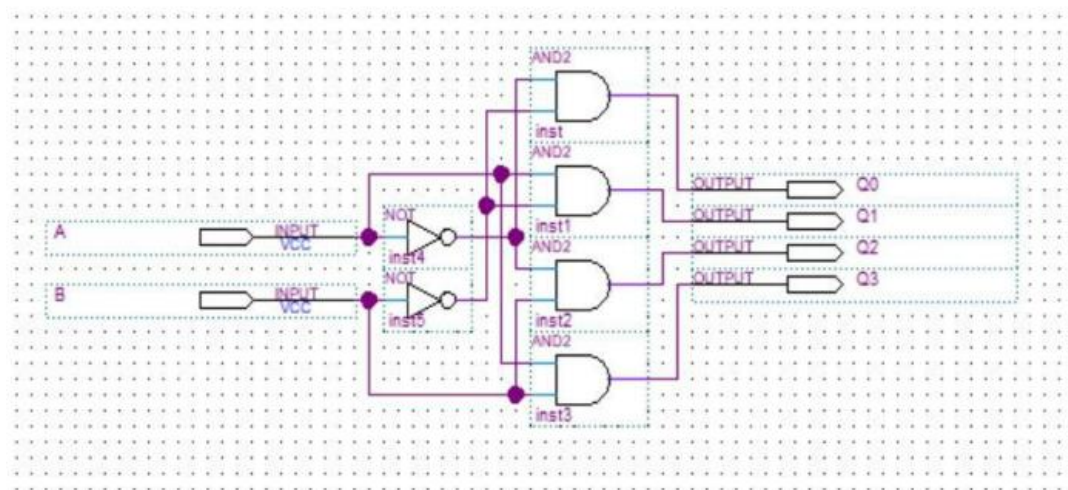
(4) 移位器



(5)ALU 整体电路

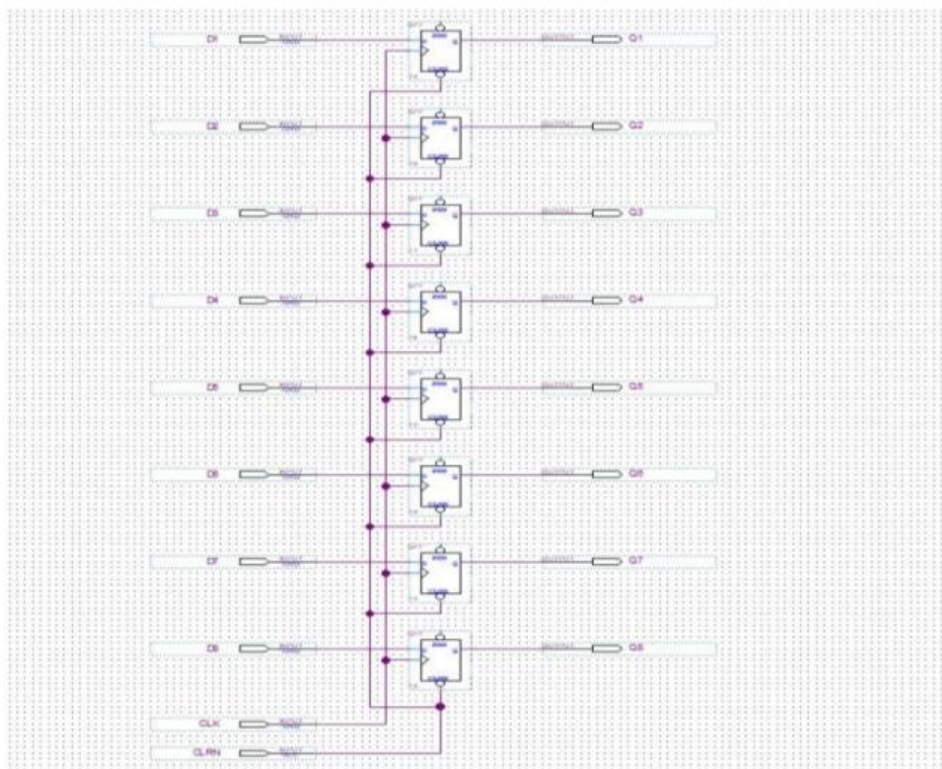


(6) 2-4 译码器

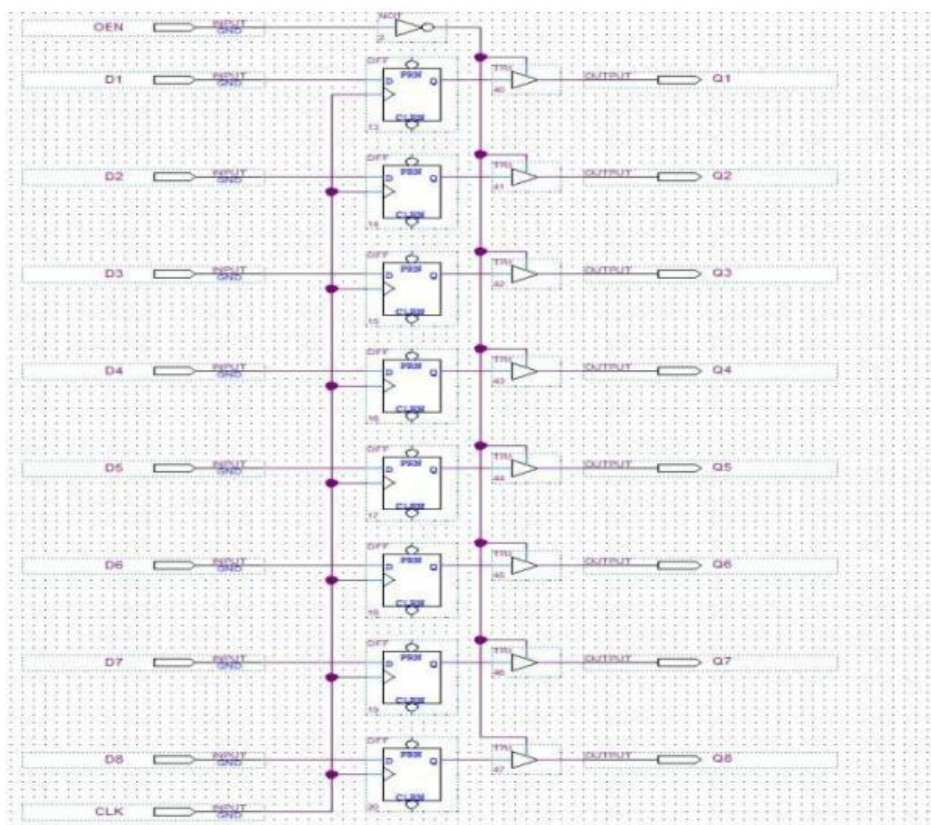


(7) 4-16 译码器

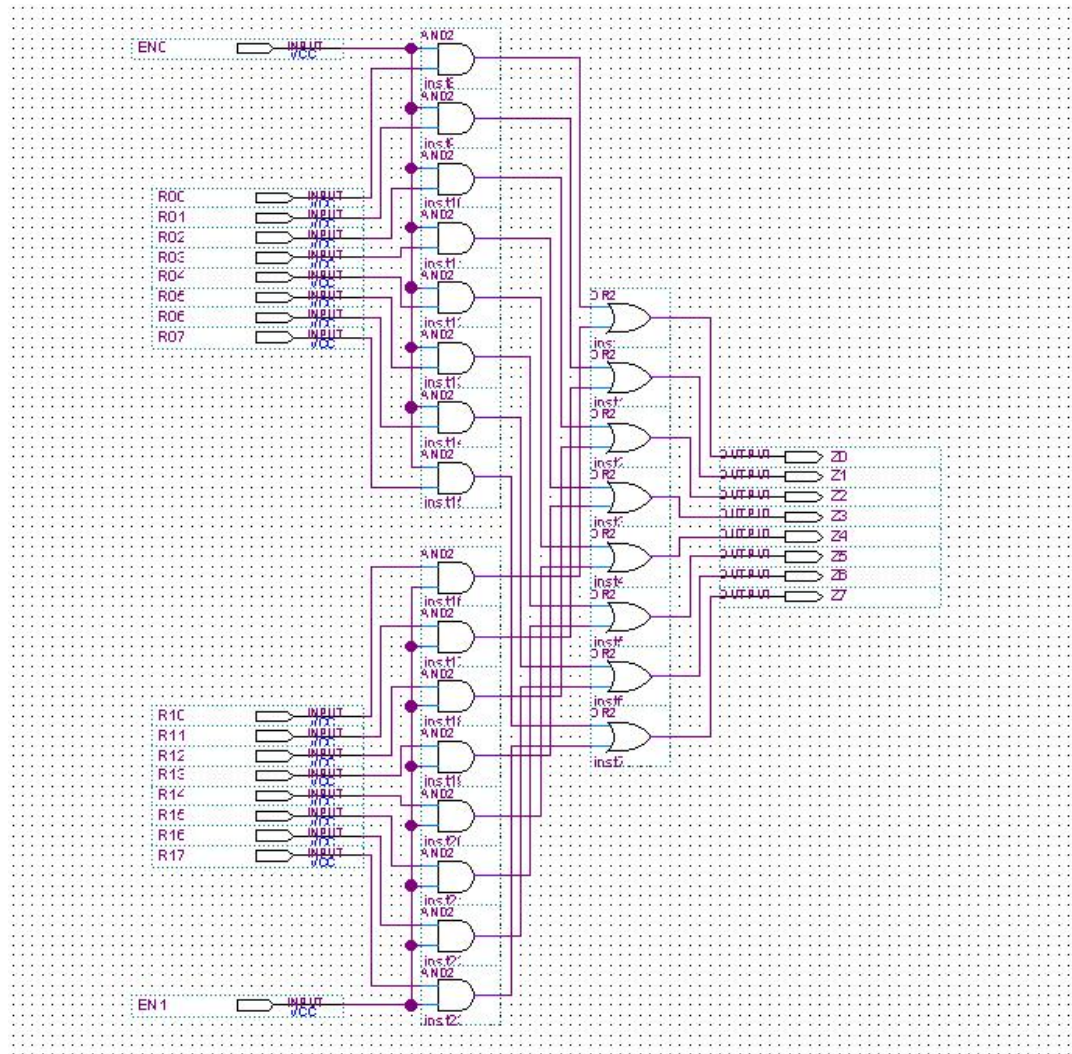
63



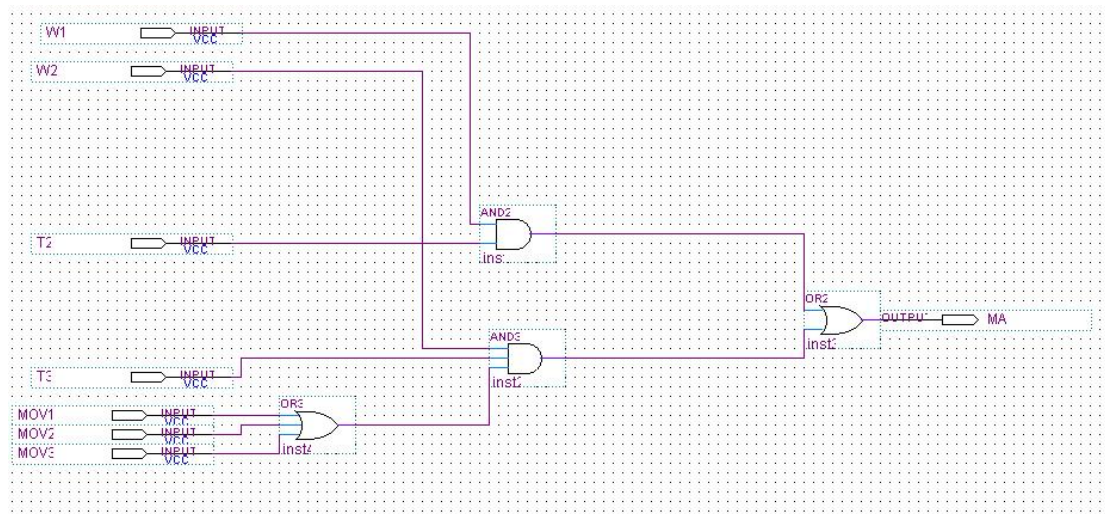
(9) R0、R1 寄存器

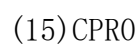
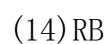
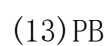


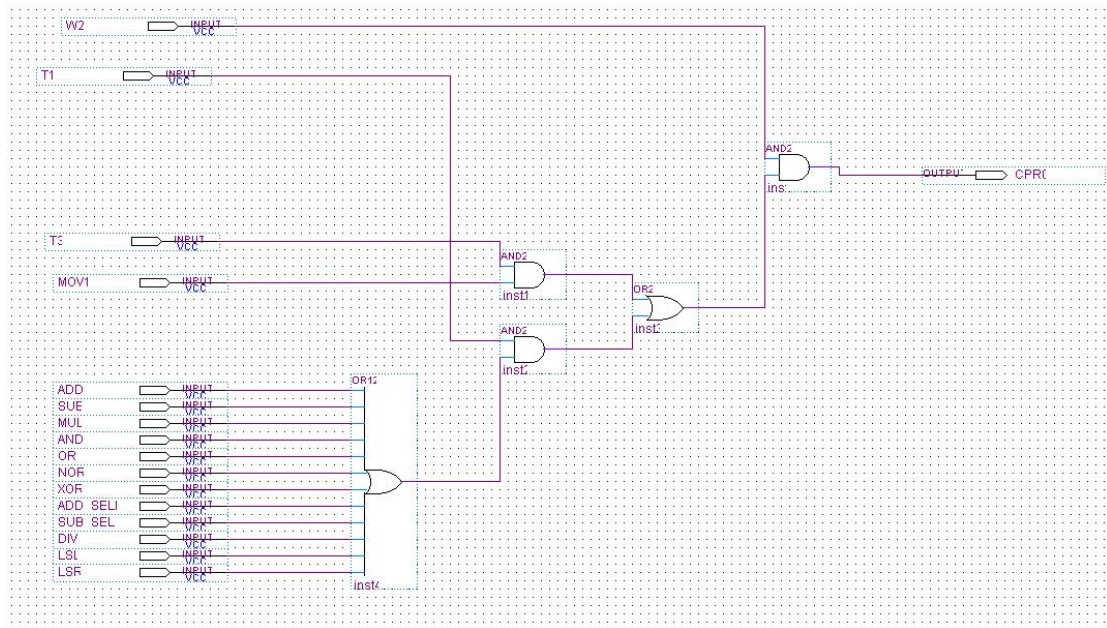
(10)2 选 1 数据选择器



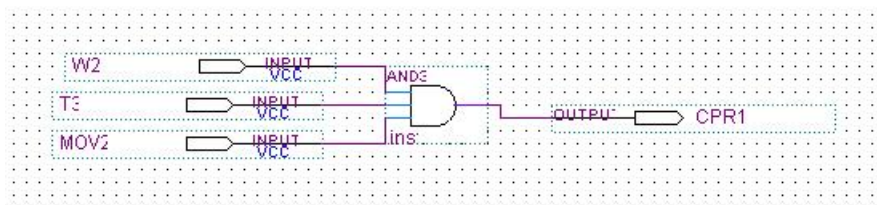
(11)MA



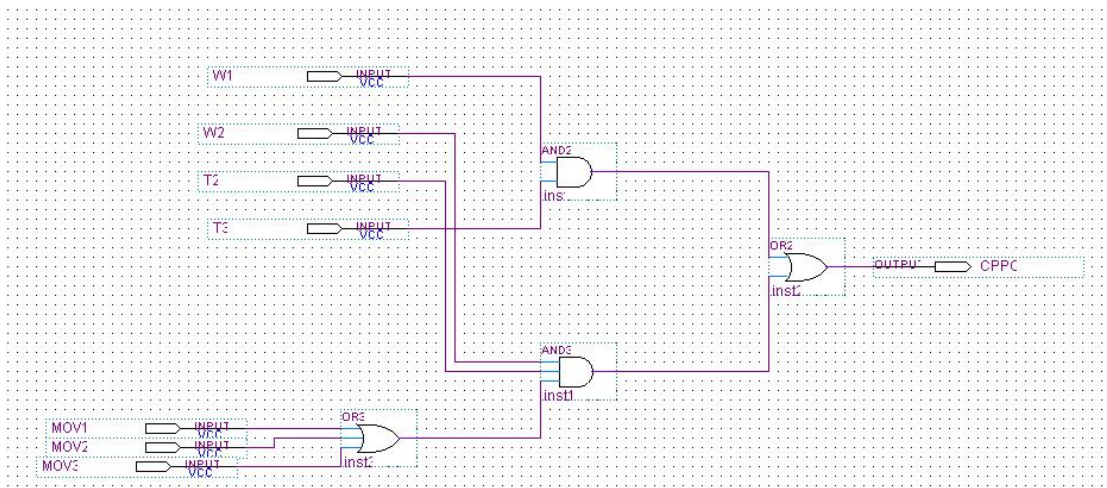




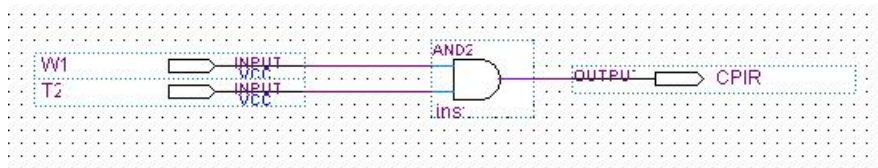
(16) CPR1



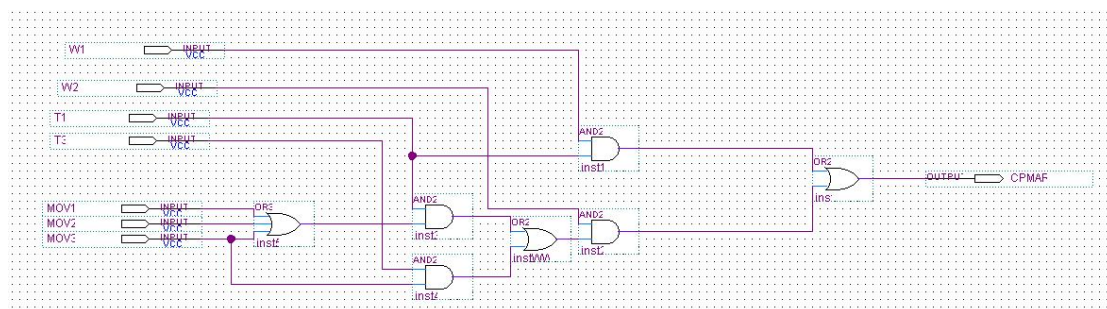
(17) CPPC



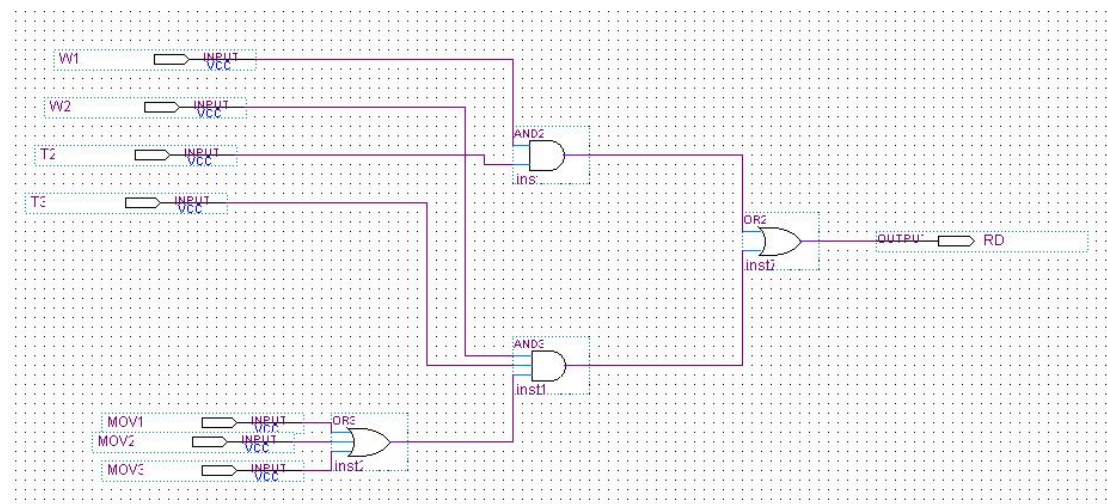
(18) CPIR



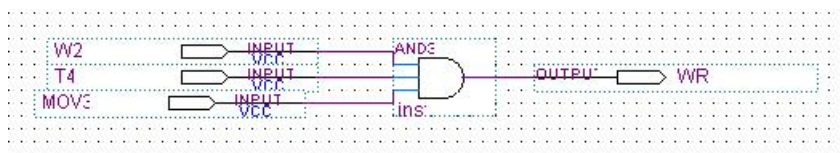
(19) CPMAR



(20) RD



(21) WR

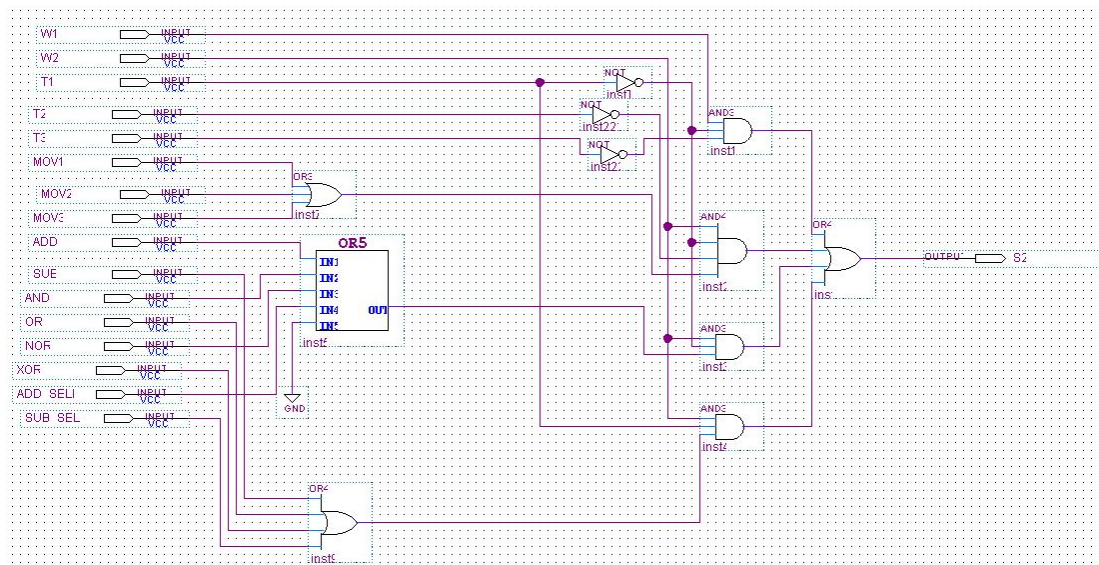


(22) G

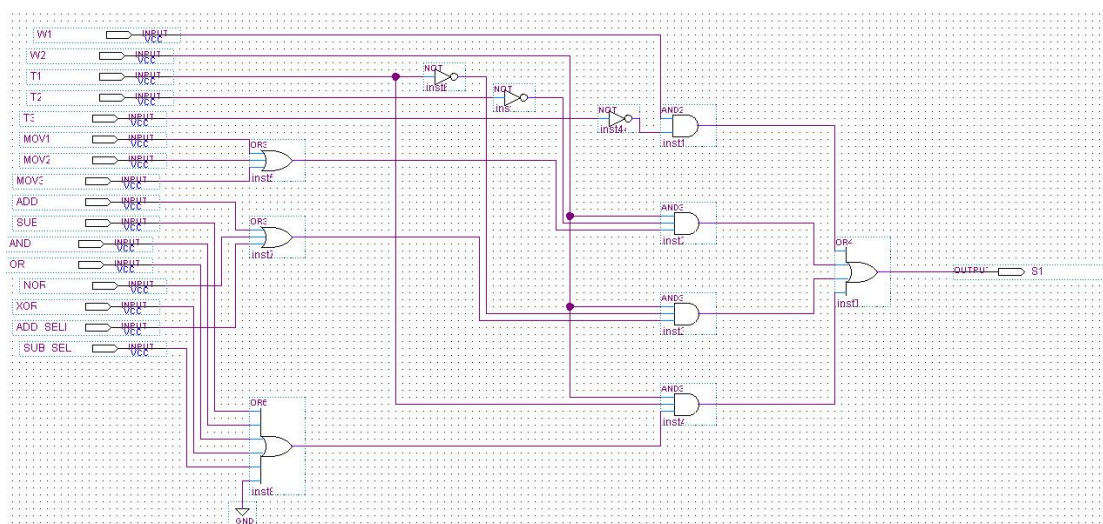
[illegible]

69

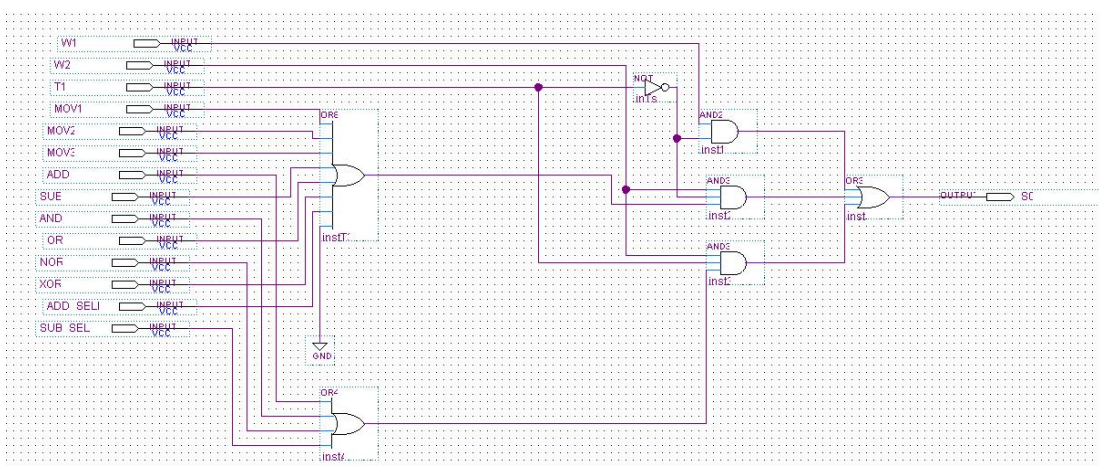
计算机组成与设计课程设计报告



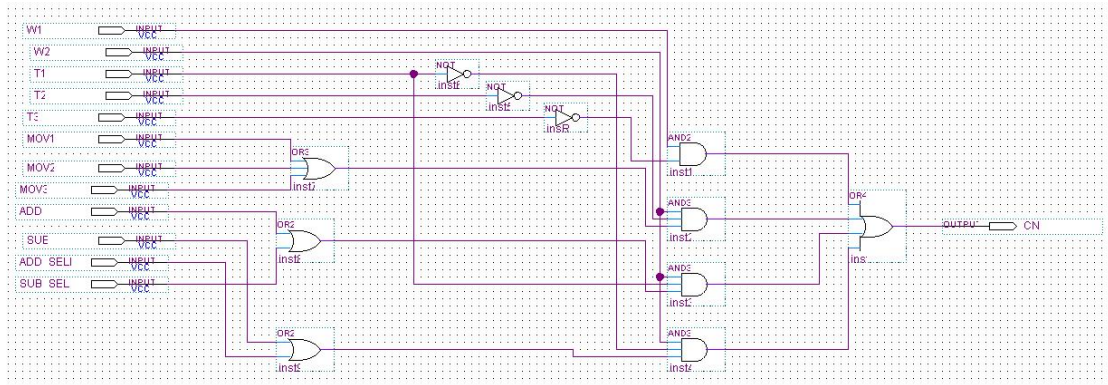
(26) S1



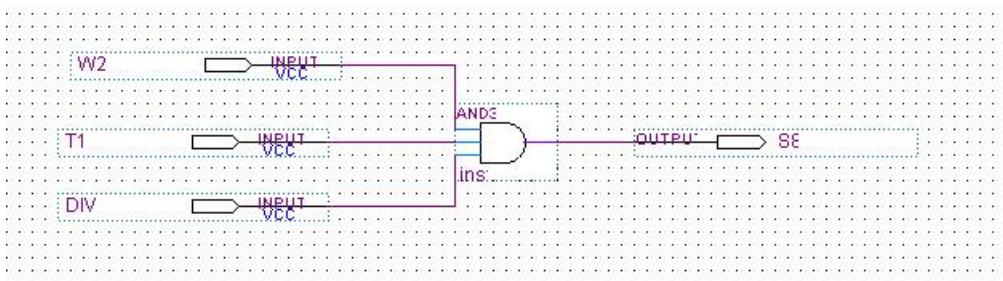
(27) S0



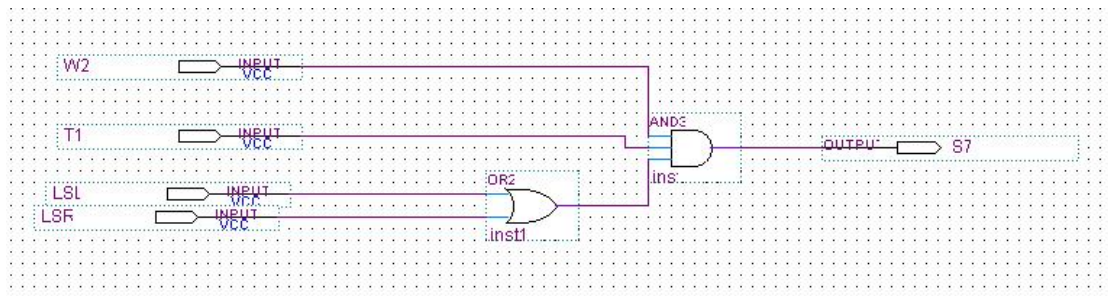
(28) CN



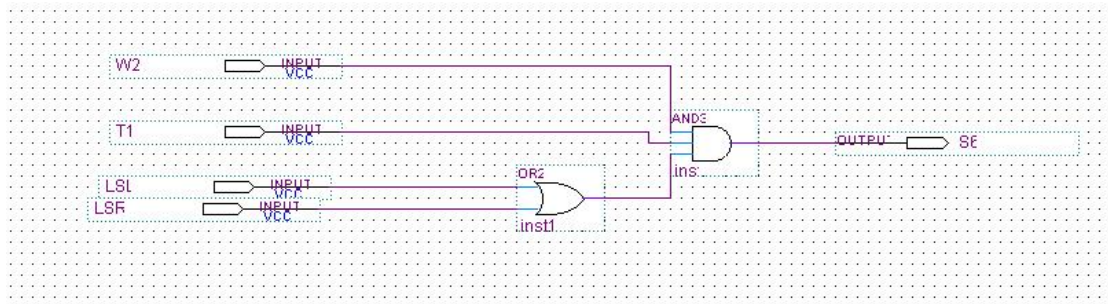
(29) S8



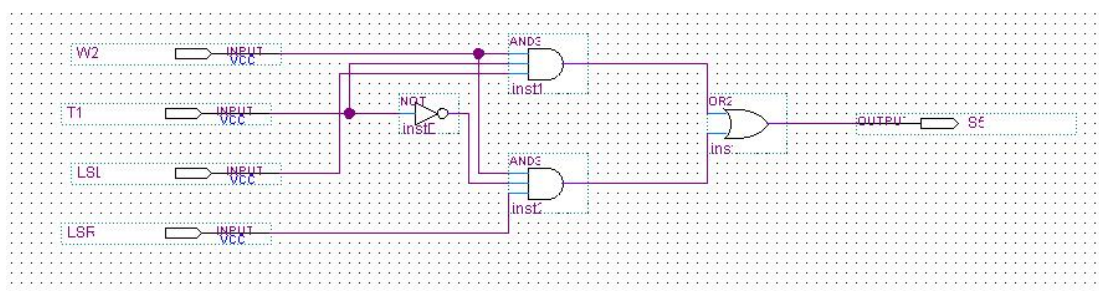
(30) S7



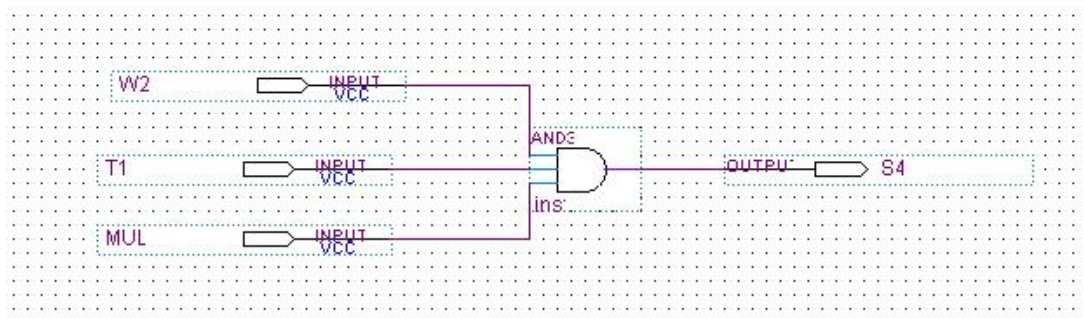
(31) S6



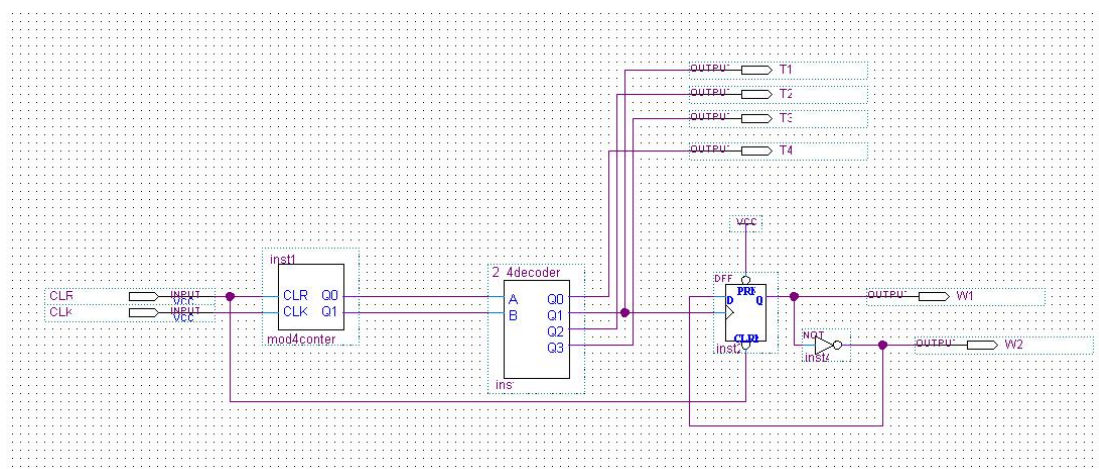
(32) S5



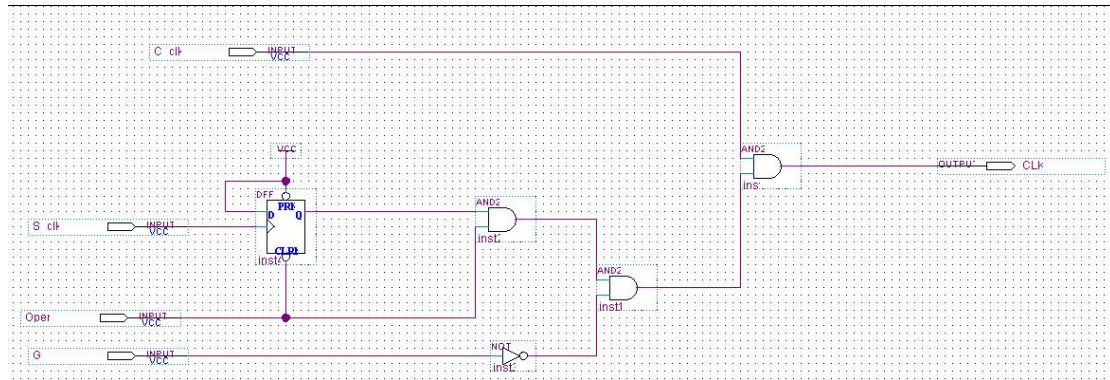
(33) S4



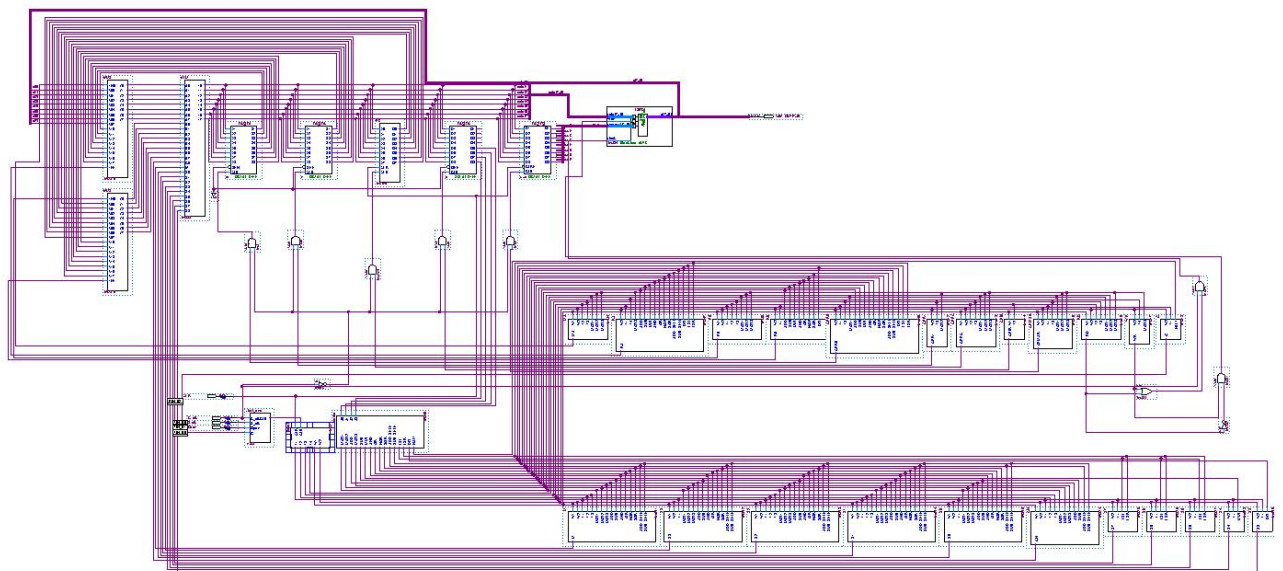
(34) 节拍发生器



(35) 启动电路



6.9 完整电路结构



七、课程设计总结

遇到的问题以及解决

问题：硬布线控制的 CU 之中，增加一条指令就会导致几乎大部分控制部件的线路发生改变。

解决：采用分层控制的思想。例如，把周期和节拍的 and 放在一层，最后每个信号再一层，这样添加指令只要做到有序更改，不会出错。或提前把所有指令写完，写出每个信号的表达式，然后一次性完成。

问题：微程序设计时 16 条指令不够怎么办？

解决：在一开始设计中操作码只有 4 位，故指令最多 16 条，而这实际上是远远不够的，故在实验中我们采用的是 5 位操作码。其实还可以采用 6 位操作码，这样可以定义 64 条指令，同时需要考虑增加 ROM 的地址线，也可以多写几个 CROM 以及 RAM，分别测试不同的指令，这样不用大改整体架构。

（3）从实验二开始便有一条指令开启很多指令信号的情况。如何保证指令的信号正确？

解决：在实验中我们将每个信号绑定在不同节拍上，以此来保证数据能按照正确的顺序传输。我们采用四节拍发生器来产生时序。时钟连续脉冲作为节拍发生器的输入。第一个节拍控制寄存器组的脉冲，第二个节拍控制 uPC 的脉冲，第三个节拍控制 ROM 的脉冲，第四个节拍控制 RAM 的脉冲。通过节拍发生器产生脉冲，不会出现时序竞争问题，比较安全。

设计的优点

微程序简单模型机设计思路清晰，实现相对完整。模型机在实现基本的加减算术运算的基础上，又实现了与、或、非、异或、或非、左移、右移、自增、自减、乘、除等功能。同时，模型机还实现了三个函数：求取平均数、计算累乘以及求解 x^y ，实验中利用变址寻址实现 do_while 循环结构。模型机时序清晰，不会产生竞争冒险，系统具有高可靠性。硬布线简单模型机设计逻辑清晰，信号设计正确，在实现基本加减算术的基础上，又实现了与、或、非、异或、或非、左移、右移、自增、自减、乘法和除法功能，由于时序由节拍发生器控制，并且我们对一些脉冲做了“特殊处理”，使得时序不会出现混乱，也确保了整个系统的高稳定性。

设计的缺点

微程序模型机和硬布线模型机的设计也存在一定的缺陷，比如由于选取的 ROM 大小为 $8 * 32$ ，导致指令操作码受限，指令条数过少，在微程序实验中只有 32 条指令，可以考虑使用更大的 ROM 来增加操作码位数，以此可以设计更多的指令。同时系统都有一定的优化空间，如采用流水线的方式来提高系统的执行效率、通过建立 cache 来减少访存次数等等。

收获与体会

经过这次《计算机组成与设计课程设计》的大实验锻炼，也相当于是自己从电路原理角度，实现了或者说模拟了一个计算机的原始模型机，包括控制器、运算器、

存储器、IO 设备等，并且是两种控制器的实现方式。也是通过这两个实验，让我从指令的相关设计到模型机的整体结构，再到控制器的不同实现有了一个完整的概念框图。

进一步明白了两种实现方式的区别：

微程序控制器的控制功能是在存放微程序存储器和存放当前正在执行的微指令的寄存器直接控制下实现的，而硬布线控制器的控制功能则由逻辑门组合实现。微程序控制器的电路比较规整，各条指令信号的差别集中在控制存储器内容上，因此，无论是增加或修改指令都只要增加或修改控制存储器内容即可。硬布线控制器的控制信号先用逻辑式列出，经化简后用电路来实现，因此，显得零乱复杂，当需要修改指令或者增加指令时就必须重新设计电路，非常麻烦而且有时甚至无法改变。

在同样的半导体工艺条件下，微程序控制的速度比硬布线控制的速度低，因为执行每条微程序指令都要从控制存储器中读取，影响执行速度。而硬布线控制逻辑主要取决于电路延时，因而在超高速机器中，对影响速度的关键部分如核心部件 CPU，往往采用硬布线逻辑实现。在一些新型计算机系统中，例如，RISC（精简指令集计算机）中，一般都选用硬布线逻辑电路。

在整个课程设计下来我的感受一句话概括就是：知其然知其所以然。

当这个课设从底层一条条线搭建起来的时候，必须每一层都很清晰。必须明白整体流程架构，才能不出方向错误。我们在一开始时并没有太理解实验，导致走了很多弯路，花了很多功夫来排错。之后我们反复理解实验，才明白每一条指令的执行流程，进而编好微程序。只有明白每一个器件的工作原理以及设计方式，才能一层层搭建起这个模型机。

硬件的学习，对于计科人来说，计组是十分重要的一门课。除此之外，计算机体系结构以及计算机系统原理跟计组都是相辅相成的。近些年来硬件行业的发展也很不错，我们从最初的学习电子电路、计算机系统原理，到后来数字逻辑、计组，再到本次的课设的融会贯通，先前课程的学习，也都为本次课设做了铺垫。这次的课设为我们以后的发展积累了一些经验和信心，也让我们对计算机组成与设计这门学科有了实践性的认识。